Using SAT Solvers

in Combinatorics, Combinatorial Geometry, and Graph Drawing

Manfred Scheucher

Boolean satisfiability problem

- Given a Boolean formula, is there an assignment such that the formula is true?

Boolean satisfiability problem

- Given a Boolean formula, is there an assignment such that the formula is true?

- NP-complete, but quite good heuristics

Why to use SAT Solvers?

- Prove/disprove existence of structure

usually faster

Why to use SAT Solvers?

- Prove/disprove existence of structure

usually faster

unsatisfiability can be certified

Why to use SAT Solvers?

- Prove/disprove existence of structure
- Find (counter)examples
- Counting occurences
- Induction base

Today's Topics

- L-shaped Point Set Embeddings of Trees
  (with Torsten Mütze)
- Orthogonal Symmetric Chain Decompositions
  (with Karl Däubel, Sven Jäger, and Torsten Mütze)
- A SAT Attack on Rota's Basis Conjecture
  (with Markus Kirchweger and Stefan Szeider)
- Disjoint Holes in Point Sets
- Universal Point Sets for Planar Graphs
  (with Hendrik Schrezenmaier and Raphael Steiner)
- ...

A Short Sage/Minisat Example

$\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$
(conjunctive normal form)

```
$ sage
...
sage: solver = SAT()
sage: solver.add_clause([1,2,3])
sage: solver.add_clause([-1,-2,-3])
sage: solver()
(None, False, False, True)
```

A Short Pycosat Example

```
$ ipython
...
In [1]: import pycosat

In [2]: CNF = [[1,2,3],[-1,-2,-3]]

In [3]: for sol in pycosat.itersolve(CNF): print sol
[-1, -2, 3]
[-1, 2, -3]
[-1, 2, 3]
[1, 2, -3]
[1, -2, 3]
[1, -2, -3]
```

Another simple example

given a graph $G = (V, E)$

for $i, j \in \binom{V}{2}$, variables X_{ij} indicates whether $ij \in E$

Another simple example

given a graph $G = (V, E)$

for $i, j \in \binom{V}{2}$, variables X_{ij} indicates whether $ij \in E$

Ramsey $R(3, 3)$: neither G nor its complement contains K_3

Another simple example

given a graph $G = (V, E)$

for $i, j \in \binom{V}{2}$, variables X_{ij} indicates whether $ij \in E$

Ramsey $R(3,3)$: neither G nor its complement contains K_3

$\forall a, b, c \in \binom{V}{3} : \neg X_{ab} \vee \neg X_{ac} \vee \neg X_{bc}$ and $X_{ab} \vee X_{ac} \vee X_{bc}$

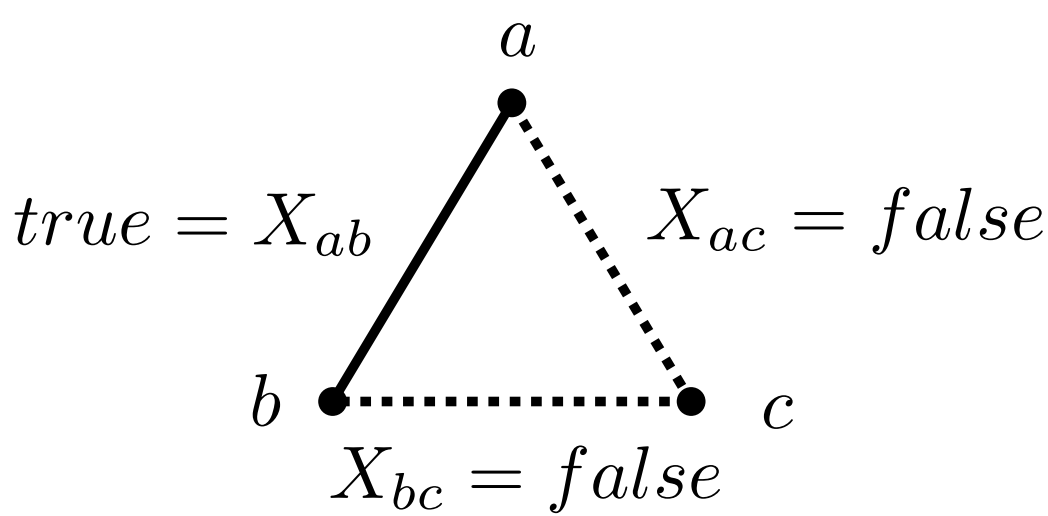
Another simple example

given a graph $G = (V, E)$

for $i, j \in \binom{V}{2}$, variables X_{ij} indicates whether $ij \in E$

Ramsey $R(3,3)$: neither G nor its complement contains K_3

$\forall a, b, c \in \binom{V}{3} : \neg X_{ab} \vee \neg X_{ac} \vee \neg X_{bc}$ and $X_{ab} \vee X_{ac} \vee X_{bc}$



Another simple example

given a graph $G = (V, E)$

for $i, j \in \binom{V}{2}$, variables X_{ij} indicates whether $ij \in E$

Ramsey $R(3,3)$: neither G nor its complement contains K_3

$\forall a, b, c \in \binom{V}{3} : \neg X_{ab} \vee \neg X_{ac} \vee \neg X_{bc}$ and $X_{ab} \vee X_{ac} \vee X_{bc}$

in python:

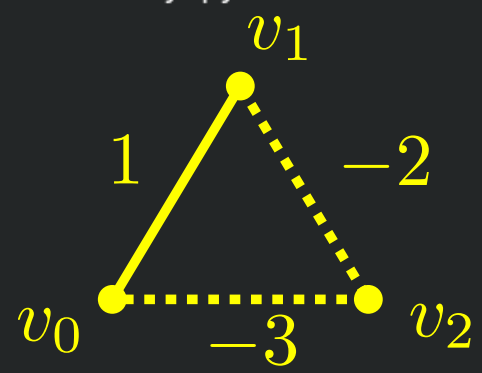
```
for a,b,c in combinations(V,3):
  cnf.append([-edge_var(a,b),-edge_var(a,c),-edge_var(b,c)])
  cnf.append([+edge_var(a,b),+edge_var(a,c),+edge_var(b,c)])
```

ramsey.py | test_chirotope.py | test_signotope.py | test_theorem3.py | signotopes.py | for a,b,c,d in combinations(range(n),4):

```python
from itertools import combinations
from pysat.formula import IDPool,CNF
from pysat.solvers import Solver
from sys import argv

n = int(argv[1])
vertices = range(n)

variables = IDPool()
edge_var = lambda a,b: variables.id((a,b))

print("edge vars:",{(a,b):edge_var(a,b) for (a,b) in combinations(vertices,2)})

cnf = CNF()
for a,b,c in combinations(vertices,3):
    cnf.append([+edge_var(a,b),+edge_var(a,c),+edge_var(b,c)])
    cnf.append([-edge_var(a,b),-edge_var(a,c),-edge_var(b,c)])

print("clauses:",cnf.clauses)

print("solutions:")
s = Solver(bootstrap_with=cnf.clauses)
for ct,sol in enumerate(s.enum_models()):
    edges = [(a,b) for (a,b) in combinations(vertices,2) if edge_var(a,b) in sol]
    print("#",ct,":",sol,"->",edges)

print("end.")
```

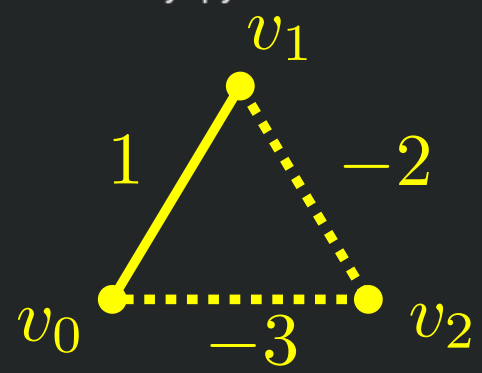
```
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 3
edge vars: {(0, 1): 1, (0, 2): 2, (1, 2): 3}
clauses: [[1, 2, 3], [-1, -2, -3]]
solutions:
# 0 : [1, -2, -3] -> [(0, 1)]
# 1 : [1, 2, -3] -> [(0, 1), (0, 2)]
# 2 : [-1, 2, 3] -> [(0, 2), (1, 2)]
# 3 : [-1, -2, 3] -> [(1, 2)]
# 4 : [-1, 2, -3] -> [(0, 2)]
# 5 : [1, -2, 3] -> [(0, 1), (1, 2)]
end.
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 5
edge vars: {(0, 1): 1, (0, 2): 2, (0, 3): 3, (0, 4): 4, (1, 2): 5, (1, 3): 6, (1, 4): 7, (2, 3): 8, (2, 4): 9, (3,
clauses: [[1, 2, 5], [-1, -2, -5], [1, 3, 6], [-1, -3, -6], [1, 4, 7], [-1, -4, -7], [2, 3, 8], [-2, -3, -8], [2,
, [-5, -6, -8], [5, 7, 9], [-5, -7, -9], [6, 7, 10], [-6, -7, -10], [8, 9, 10], [-8, -9, -10]]
solutions:
# 0 : [-1, -2, 3, 4, 5, -6, 7, 8, -9, -10] -> [(0, 3), (0, 4), (1, 2), (1, 4), (2, 3)]
# 1 : [-1, 2, -3, 4, -5, 6, 7, 8, -9, -10] -> [(0, 2), (0, 4), (1, 3), (1, 4), (2, 3)]
# 2 : [1, -2, 3, -4, -5, -6, 7, 8, 9, -10] -> [(0, 1), (0, 3), (1, 4), (2, 3), (2, 4)]
# 3 : [1, -2, -3, 4, -5, 6, -7, 8, 9, -10] -> [(0, 1), (0, 4), (1, 3), (2, 3), (2, 4)]
# 4 : [1, -2, -3, 4, 5, -6, -7, 8, -9, 10] -> [(0, 1), (0, 4), (1, 2), (2, 3), (3, 4)]
# 5 : [1, 2, -3, -4, -5, -6, 7, 8, -9, 10] -> [(0, 1), (0, 2), (1, 4), (2, 3), (3, 4)]
# 6 : [1, 2, -3, -4, -5, 6, -7, -8, 9, 10] -> [(0, 1), (0, 2), (1, 3), (2, 4), (3, 4)]
# 7 : [-1, 2, -3, 4, 5, 6, -7, -8, -9, 10] -> [(0, 2), (0, 4), (1, 2), (1, 3), (3, 4)]
# 8 : [-1, -2, 3, 4, 5, 6, -7, -8, 9, -10] -> [(0, 3), (0, 4), (1, 2), (1, 3), (2, 4)]
# 9 : [-1, 2, 3, -4, -5, 6, 7, -8, 9, -10] -> [(0, 2), (0, 3), (1, 3), (1, 4), (2, 4)]
# 10 : [-1, 2, 3, -4, 5, -6, 7, -8, -9, 10] -> [(0, 2), (0, 3), (1, 2), (1, 4), (3, 4)]
# 11 : [1, -2, 3, -4, 5, -6, -7, -8, 9, 10] -> [(0, 1), (0, 3), (1, 2), (2, 4), (3, 4)]
end.
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 6
edge vars: {(0, 1): 1, (0, 2): 2, (0, 3): 3, (0, 4): 4, (0, 5): 5, (1, 2): 6, (1, 3): 7, (1, 4): 8, (1, 5): 9, (2,
(4, 5): 15}
clauses: [[1, 2, 6], [-1, -2, -6], [1, 3, 7], [-1, -3, -7], [1, 4, 8], [-1, -4, -8], [1, 5, 9], [-1, -5, -9], [2,
12], [-2, -5, -12], [3, 4, 13], [-3, -4, -13], [3, 5, 14], [-3, -5, -14], [4, 5, 15], [-4, -5, -15], [6, 7, 10], [
 -9, -12], [7, 8, 13], [-7, -8, -13], [7, 9, 14], [-7, -9, -14], [8, 9, 15], [-8, -9, -15], [10, 11, 13], [-10, -1
1, -12, -15], [13, 14, 15], [-13, -14, -15]]
solutions:
end.
manfred@mscheuch:~/SoCG/structures/examples$
```

```
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 3
edge vars: {(0, 1): 1, (0, 2): 2, (1, 2): 3}
clauses: [[1, 2, 3], [-1, -2, -3]]
solutions:
# 0 : [1, -2, -3] -> [(0, 1)]
# 1 : [1, 2, -3] -> [(0, 1), (0, 2)]
# 2 : [-1, 2, 3] -> [(0, 2), (1, 2)]
# 3 : [-1, -2, 3] -> [(1, 2)]
# 4 : [-1, 2, -3] -> [(0, 2)]
# 5 : [1, -2, 3] -> [(0, 1), (1, 2)]
end.
```

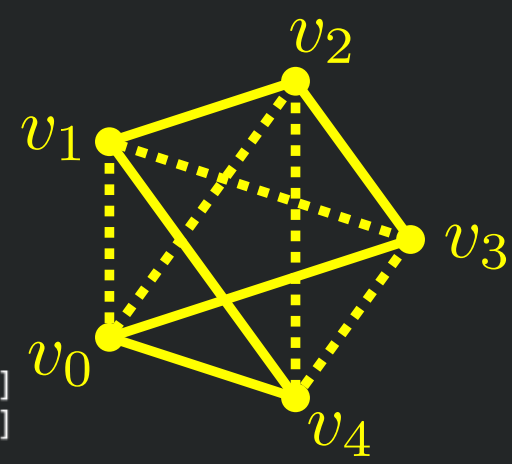



```
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 5
edge vars: {(0, 1): 1, (0, 2): 2, (0, 3): 3, (0, 4): 4, (1, 2): 5, (1, 3): 6, (1, 4): 7, (2, 3): 8, (2, 4): 9, (3,
clauses: [[1, 2, 5], [-1, -2, -5], [1, 3, 6], [-1, -3, -6], [1, 4, 7], [-1, -4, -7], [2, 3, 8], [-2, -3, -8], [2,
, [-5, -6, -8], [5, 7, 9], [-5, -7, -9], [6, 7, 10], [-6, -7, -10], [8, 9, 10], [-8, -9, -10]]
solutions:
# 0 : [-1, -2, 3, 4, 5, -6, 7, 8, -9, -10] -> [(0, 3), (0, 4), (1, 2), (1, 4), (2, 3)]
# 1 : [-1, 2, -3, 4, -5, 6, 7, 8, -9, -10] -> [(0, 2), (0, 4), (1, 3), (1, 4), (2, 3)]
# 2 : [1, -2, 3, -4, -5, -6, 7, 8, 9, -10] -> [(0, 1), (0, 3), (1, 4), (2, 3), (2, 4)]
# 3 : [1, -2, -3, 4, -5, 6, -7, 8, 9, -10] -> [(0, 1), (0, 4), (1, 3), (2, 3), (2, 4)]
# 4 : [1, -2, -3, 4, 5, -6, -7, 8, -9, 10] -> [(0, 1), (0, 4), (1, 2), (2, 3), (3, 4)]
# 5 : [1, 2, -3, -4, -5, -6, 7, 8, -9, 10] -> [(0, 1), (0, 2), (1, 4), (2, 3), (3, 4)]
# 6 : [1, 2, -3, -4, -5, 6, -7, -8, 9, 10] -> [(0, 1), (0, 2), (1, 3), (2, 4), (3, 4)]
# 7 : [-1, 2, -3, 4, 5, 6, -7, -8, -9, 10] -> [(0, 2), (0, 4), (1, 2), (1, 3), (3, 4)]
# 8 : [-1, -2, 3, 4, 5, 6, -7, -8, 9, -10] -> [(0, 3), (0, 4), (1, 2), (1, 3), (2, 4)]
# 9 : [-1, 2, 3, -4, -5, 6, 7, -8, 9, -10] -> [(0, 2), (0, 3), (1, 3), (1, 4), (2, 4)]
# 10 : [-1, 2, 3, -4, 5, -6, 7, -8, -9, 10] -> [(0, 2), (0, 3), (1, 2), (1, 4), (3, 4)]
# 11 : [1, -2, 3, -4, 5, -6, -7, -8, 9, 10] -> [(0, 1), (0, 3), (1, 2), (2, 4), (3, 4)]
end.
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 6
edge vars: {(0, 1): 1, (0, 2): 2, (0, 3): 3, (0, 4): 4, (0, 5): 5, (1, 2): 6, (1, 3): 7, (1, 4): 8, (1, 5): 9, (2,
(4, 5): 15}
clauses: [[1, 2, 6], [-1, -2, -6], [1, 3, 7], [-1, -3, -7], [1, 4, 8], [-1, -4, -8], [1, 5, 9], [-1, -5, -9], [2,
12], [-2, -5, -12], [3, 4, 13], [-3, -4, -13], [3, 5, 14], [-3, -5, -14], [4, 5, 15], [-4, -5, -15], [6, 7, 10], [
 -9, -12], [7, 8, 13], [-7, -8, -13], [7, 9, 14], [-7, -9, -14], [8, 9, 15], [-8, -9, -15], [10, 11, 13], [-10, -1
1, -12, -15], [13, 14, 15], [-13, -14, -15]]
solutions:
end.
manfred@mscheuch:~/SoCG/structures/examples$
```

```
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 3
edge vars: {(0, 1): 1, (0, 2): 2, (1, 2): 3}
clauses: [[1, 2, 3], [-1, -2, -3]]
solutions:
# 0 : [1, -2, -3] -> [(0, 1)]
# 1 : [1, 2, -3] -> [(0, 1), (0, 2)]
# 2 : [-1, 2, 3] -> [(0, 2), (1, 2)]
# 3 : [-1, -2, 3] -> [(1, 2)]
# 4 : [-1, 2, -3] -> [(0, 2)]
# 5 : [1, -2, 3] -> [(0, 1), (1, 2)]
end.
```

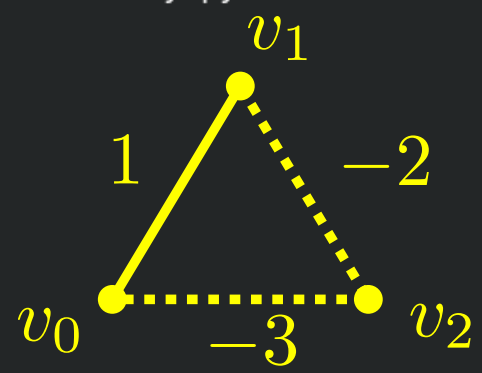



```
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 5
edge vars: {(0, 1): 1, (0, 2): 2, (0, 3): 3, (0, 4): 4, (1, 2): 5, (1, 3): 6, (1, 4): 7, (2, 3): 8, (2, 4): 9, (3,
clauses: [[1, 2, 5], [-1, -2, -5], [1, 3, 6], [-1, -3, -6], [1, 4, 7], [-1, -4, -7], [2, 3, 8], [-2, -3, -8], [2,
, [-5, -6, -8], [5, 7, 9], [-5, -7, -9], [6, 7, 10], [-6, -7, -10], [8, 9, 10], [-8, -9, -10]]
solutions:
# 0 : [-1, -2, 3, 4, 5, -6, 7, 8, -9, -10] -> [(0, 3), (0, 4), (1, 2), (1, 4), (2, 3)]
# 1 : [-1, 2, -3, 4, -5, 6, 7, 8, -9, -10] -> [(0, 2), (0, 4), (1, 3), (1, 4), (2, 3)]
# 2 : [1, -2, 3, -4, -5, -6, 7, 8, 9, -10] -> [(0, 1), (0, 3), (1, 4), (2, 3), (2, 4)]
# 3 : [1, -2, -3, 4, -5, 6, -7, 8, 9, -10] -> [(0, 1), (0, 4), (1, 3), (2, 3), (2, 4)]
# 4 : [1, -2, -3, 4, 5, -6, -7, 8, -9, 10] -> [(0, 1), (0, 4), (1, 2), (2, 3), (3, 4)]
# 5 : [1, 2, -3, -4, -5, -6, 7, 8, -9, 10] -> [(0, 1), (0, 2), (1, 4), (2, 3), (3, 4)]
# 6 : [1, 2, -3, -4, -5, 6, -7, -8, 9, 10] -> [(0, 1), (0, 2), (1, 3), (2, 4), (3, 4)]
# 7 : [-1, 2, -3, 4, 5, 6, -7, -8, -9, 10] -> [(0, 2), (0, 4), (1, 2), (1, 3), (3, 4)]
# 8 : [-1, -2, 3, 4, 5, 6, -7, -8, 9, -10] -> [(0, 3), (0, 4), (1, 2), (1, 3), (2, 4)]
# 9 : [-1, 2, 3, -4, -5, 6, 7, -8, 9, -10] -> [(0, 2), (0, 3), (1, 3), (1, 4), (2, 4)]
# 10 : [-1, 2, 3, -4, 5, -6, 7, -8, -9, 10] -> [(0, 2), (0, 3), (1, 2), (1, 4), (3, 4)]
# 11 : [1, -2, 3, -4, 5, -6, -7, -8, 9, 10] -> [(0, 1), (0, 3), (1, 2), (2, 4), (3, 4)]
end.
```

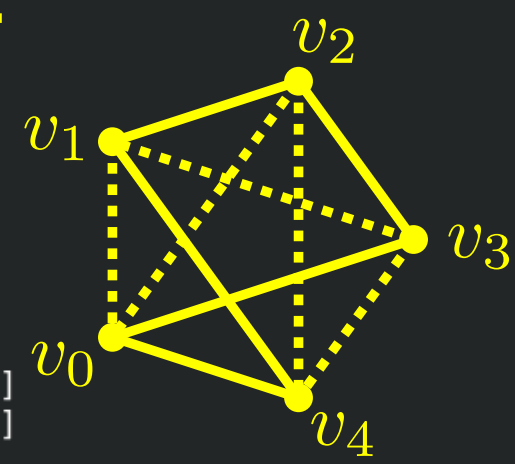



```
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 6
edge vars: {(0, 1): 1, (0, 2): 2, (0, 3): 3, (0, 4): 4, (0, 5): 5, (1, 2): 6, (1, 3): 7, (1, 4): 8, (1, 5): 9, (2,
(4, 5): 15}
clauses: [[1, 2, 6], [-1, -2, -6], [1, 3, 7], [-1, -3, -7], [1, 4, 8], [-1, -4, -8], [1, 5, 9], [-1, -5, -9], [2,
12], [-2, -5, -12], [3, 4, 13], [-3, -4, -13], [3, 5, 14], [-3, -5, -14], [4, 5, 15], [-4, -5, -15], [6, 7, 10], [
 -9, -12], [7, 8, 13], [-7, -8, -13], [7, 9, 14], [-7, -9, -14], [8, 9, 15], [-8, -9, -15], [10, 11, 13], [-10, -1
1, -12, -15], [13, 14, 15], [-13, -14, -15]]
solutions:
end.
manfred@mscheuch:~/SoCG/structures/examples$
```

```
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 3
edge vars: {(0, 1): 1, (0, 2): 2, (1, 2): 3}
clauses: [[1, 2, 3], [-1, -2, -3]]
solutions:
# 0 : [1, -2, -3] -> [(0, 1)]
# 1 : [1, 2, -3] -> [(0, 1), (0, 2)]
# 2 : [-1, 2, 3] -> [(0, 2), (1, 2)]
# 3 : [-1, -2, 3] -> [(1, 2)]
# 4 : [-1, 2, -3] -> [(0, 2)]
# 5 : [1, -2, 3] -> [(0, 1), (1, 2)]
end.
```




```
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 5
edge vars: {(0, 1): 1, (0, 2): 2, (0, 3): 3, (0, 4): 4, (1, 2): 5, (1, 3): 6, (1, 4): 7, (2, 3): 8, (2, 4): 9, (3,
clauses: [[1, 2, 5], [-1, -2, -5], [1, 3, 6], [-1, -3, -6], [1, 4, 7], [-1, -4, -7], [2, 3, 8], [-2, -3, -8], [2,
, [-5, -6, -8], [5, 7, 9], [-5, -7, -9], [6, 7, 10], [-6, -7, -10], [8, 9, 10], [-8, -9, -10]]
solutions:
# 0 : [-1, -2, 3, 4, 5, -6, 7, 8, -9, -10] -> [(0, 3), (0, 4), (1, 2), (1, 4), (2, 3)]
# 1 : [-1, 2, -3, 4, -5, 6, 7, 8, -9, -10] -> [(0, 2), (0, 4), (1, 3), (1, 4), (2, 3)]
# 2 : [1, -2, 3, -4, -5, -6, 7, 8, 9, -10] -> [(0, 1), (0, 3), (1, 4), (2, 3), (2, 4)]
# 3 : [1, -2, -3, 4, -5, 6, -7, 8, 9, -10] -> [(0, 1), (0, 4), (1, 3), (2, 3), (2, 4)]
# 4 : [1, -2, -3, 4, 5, -6, -7, 8, -9, 10] -> [(0, 1), (0, 4), (1, 2), (2, 3), (3, 4)]
# 5 : [1, 2, -3, -4, -5, -6, 7, 8, -9, 10] -> [(0, 1), (0, 2), (1, 4), (2, 3), (3, 4)]
# 6 : [1, 2, -3, -4, -5, 6, -7, -8, 9, 10] -> [(0, 1), (0, 2), (1, 3), (2, 4), (3, 4)]
# 7 : [-1, 2, -3, 4, 5, 6, -7, -8, -9, 10] -> [(0, 2), (0, 4), (1, 2), (1, 3), (3, 4)]
# 8 : [-1, -2, 3, 4, 5, 6, -7, -8, 9, -10] -> [(0, 3), (0, 4), (1, 2), (1, 3), (2, 4)]
# 9 : [-1, 2, 3, -4, -5, 6, 7, -8, 9, -10] -> [(0, 2), (0, 3), (1, 3), (1, 4), (2, 4)]
# 10 : [-1, 2, 3, -4, 5, -6, 7, -8, -9, 10] -> [(0, 2), (0, 3), (1, 2), (1, 4), (3, 4)]
# 11 : [1, -2, 3, -4, 5, -6, -7, -8, 9, 10] -> [(0, 1), (0, 3), (1, 2), (2, 4), (3, 4)]
end.
```




```
manfred@mscheuch:~/SoCG/structures/examples$ python ramsey.py 6
edge vars: {(0, 1): 1, (0, 2): 2, (0, 3): 3, (0, 4): 4, (0, 5): 5, (1, 2): 6, (1, 3): 7, (1, 4): 8, (1, 5): 9, (2,
(4, 5): 15}
clauses: [[1, 2, 6], [-1, -2, -6], [1, 3, 7], [-1, -3, -7], [1, 4, 8], [-1, -4, -8], [1, 5, 9], [-1, -5, -9], [2,
12], [-2, -5, -12], [3, 4, 13], [-3, -4, -13], [3, 5, 14], [-3, -5, -14], [4, 5, 15], [-4, -5, -15], [6, 7, 10], [
 -9, -12], [7, 8, 13], [-7, -8, -13], [7, 9, 14], [-7, -9, -14], [8, 9, 15], [-8, -9, -15], [10, 11, 13], [-10, -1
1, -12, -15], [13, 14, 15], [-13, -14, -15]]
solutions:
end.
manfred@mscheuch:~/SoCG/structures/examples$
```

no solutions $\Rightarrow R(3,3) = 6$

More on Graphs

- dynamic symmetry breaking (Kirchweger+Szeider'21)

More on Graphs

- dynamic symmetry breaking (Kirchweger+Szeider'21)
- framework for (planar) (directed) graphs in preparation

L-Shaped Point Set Embeddings

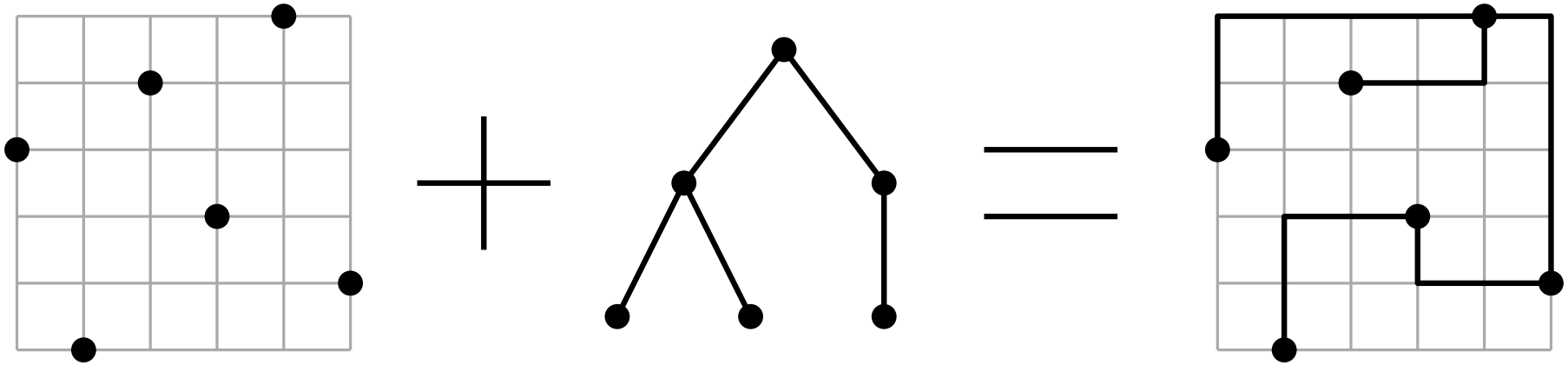
joint work with Torsten Mütze

L-Shaped Point Set Embeddings

T ... tree on n vertices

P ... set of m points

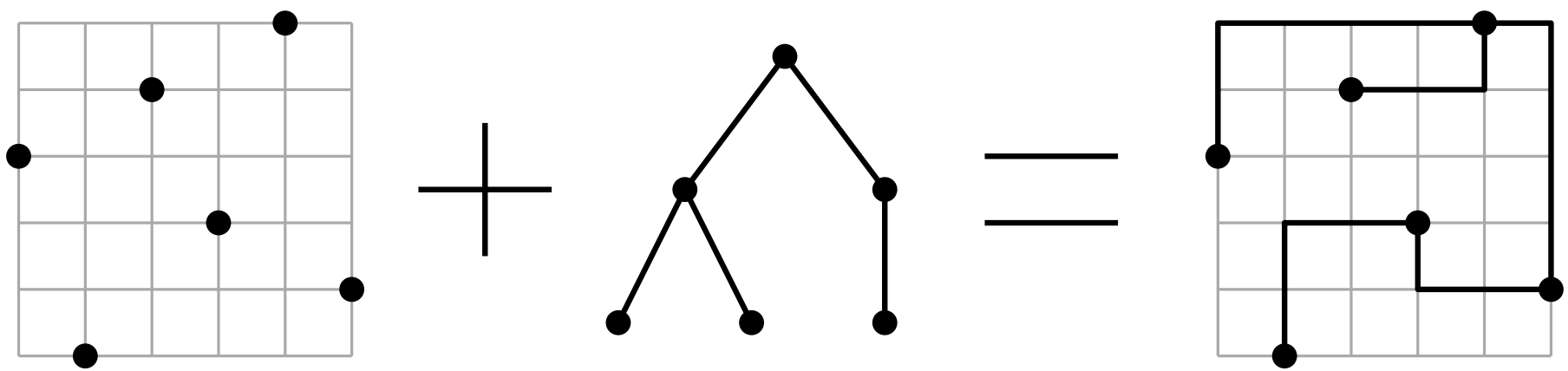
- vertices of T drawn as points of P
- edges drawn as unions of two axis parallel line segments



L-Shaped Point Set Embeddings

$f(T)$ ... minimum number m s.t. tree T admits a planar L-shaped embedding in any set of m points

$$f_d(n) := \max_{\substack{T \text{ : tree on } n \text{ vertices} \\ \text{max. deg. } \Delta(T) \leq d}} f(T)$$



L-Shaped Point Set Embeddings

$f(T)$ ... minimum number m s.t. tree T admits a planar L-shaped embedding in any set of m points

$$f_d(n) := \max_{\substack{T \text{ : tree on } n \text{ vertices} \\ \text{max. deg. } \Delta(T) \leq d}} f(T)$$

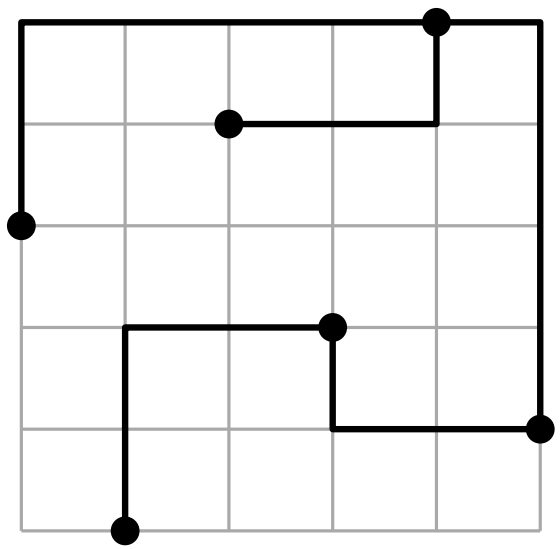
- $f_4(n) \leq n^2$ [Di Giacomo, Frati, Fulek, Grilli, Krug '13]
- $f_4(n) \leq O(n^{1.58})$ [S.'15, Aichholzer-Hackl-S.'16]
- $f_3(n) \leq O(n^{1.22})$, $f_4(n) \leq O(n^{1.55})$ [Biedl et al.'17]
- no non-trivial lower bound ($\geq n$ is trivial)

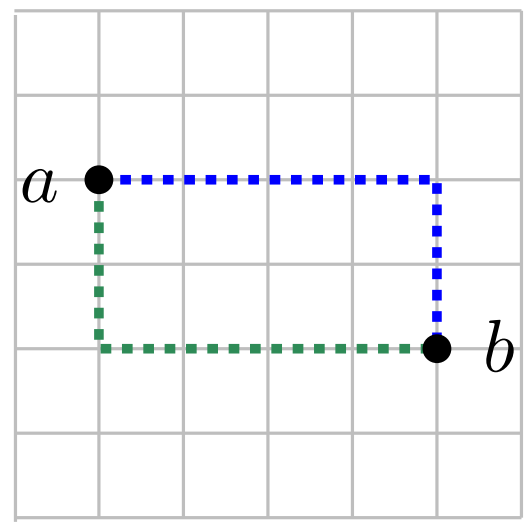
SAT Model

- T ... tree on vertices $\{v_1, \ldots, v_n\}$
- P ... point set $\{P_1, \ldots, P_n\}$
- formulate Boolean satisfiability instance:
  $\exists$ solution iff T admits an L-shaped embedding in P

SAT Model: Variables

- $M_{i,j}$ ... vertex v_i is mapped to point P_j
- $H_{a,b}$ ... edge ab is connected horizontally to a





SAT Model: Clauses

- Injective mapping V to P

  every vertex v_i has to be mapped:

$$\bigvee_j M_{i,j}$$

  no two vertices v_{i_1}, v_{i_2} are mapped to the same point:

$$\neg M_{i_1,j} \vee \neg M_{i_2,j}$$

SAT Model: Clauses

- Injective mapping V to P

every vertex v_i has to be mapped:

$$\bigvee_j M_{i,j}$$

no two vertices v_{i_1}, v_{i_2} are mapped to the same point:

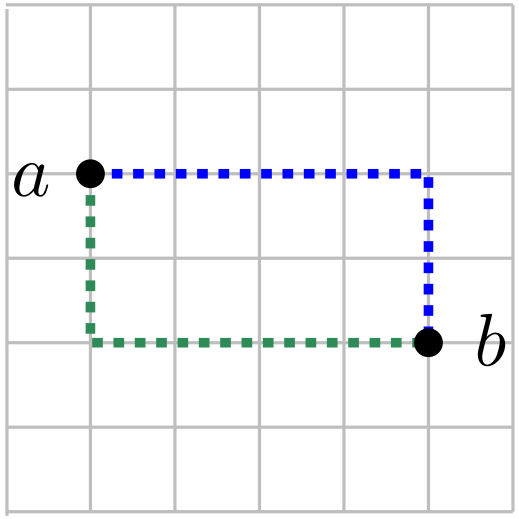
$$\neg M_{i_1,j} \vee \neg M_{i_2,j}$$

```
for i in range(n):
    CNF.append([M(i,j) for j in range(n)])

for j in range(n):
    for i1,i2 in combinations(range(n),2):
        CNF.append([-M(i1,j),-M(i2,j)])
```

SAT Model: Clauses

- Injective mapping V to P
- L-shaped edges:
  ab connects either vertically or horizontally to a (and b)

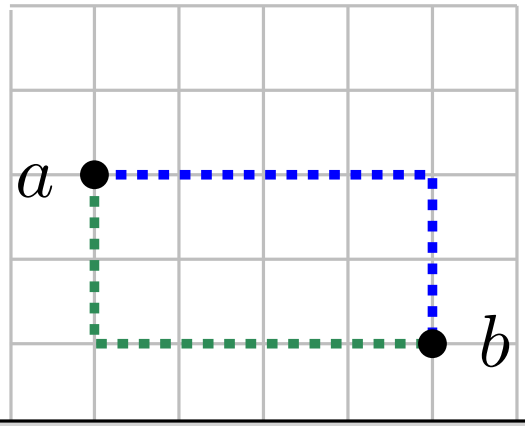
$$H_{a,b} \vee H_{b,a}, \quad \neg H_{a,b} \vee \neg H_{b,a}$$



SAT Model: Clauses

- Injective mapping V to P
- L-shaped edges:
  ab connects either vertically or horizontally to a (and b)

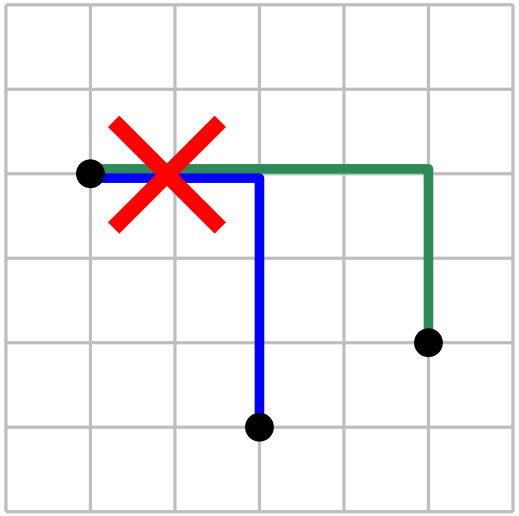
$$H_{a,b} \vee H_{b,a}, \quad \neg H_{a,b} \vee \neg H_{b,a}$$




```
for (a,b) in E:
    CNF.append([H(a,b),H(b,a)])
    CNF.append([-H(a,b),-H(b,a)])
```

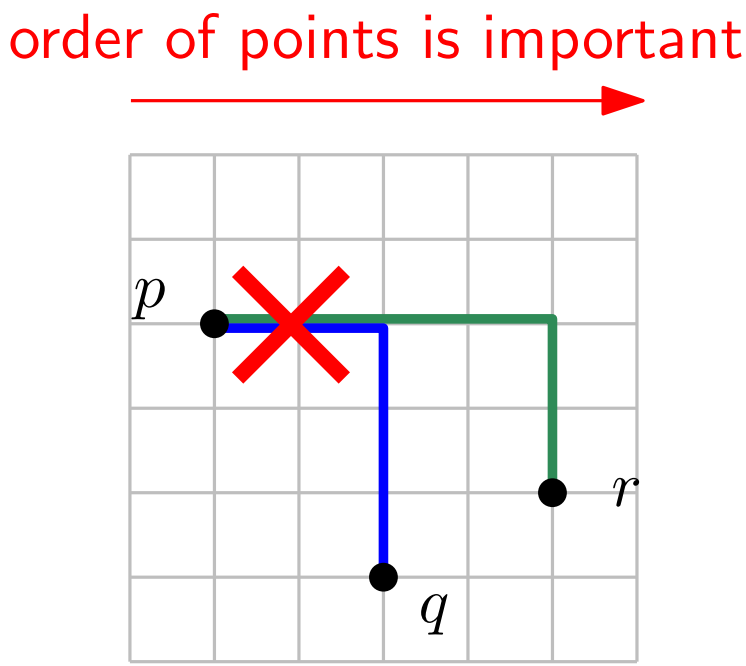
SAT Model: Clauses

- Injective mapping V to P
- L-shaped edges:
  ab connects either vertically or horizontally to a (and b)
- No overlapping edges



SAT Model: Clauses

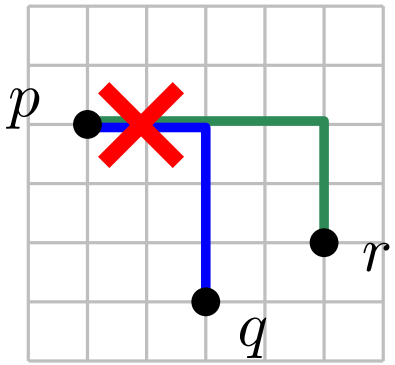
- Injective mapping V to P
- L-shaped edges:
  ab connects either vertically or horizontally to a (and b)
- No overlapping edges



SAT Model: Clauses

- Injective mapping V to P
- L-shaped edges:
  ab connects either vertically or horizontally to a (and b)
- No overlapping edges

  If p left of q and r, ← only depends on point set
  and $v_a \to p$, $v_b \to q$, $v_c \to r$, ← only depends on variables
  then not both edges ab and ac horizonally connected



SAT Model: Clauses

- Injective mapping V to P
- L-shaped edges:
  ab connects either vertically or horizontally to a (and b)
- No overlapping edges

  only depends on point set

  If p left of q and r,

  only depends on variables

  and $v_a \to p$, $v_b \to q$, $v_c \to r$,
  then not both edges ab and ac horizonally connected

  For each three points p, q, r with p left of q and r:

$$\neg M_{a,p} \vee \neg M_{b,q} \vee \neg M_{c,r} \vee \neg H_{a,b} \vee \neg H_{a,c}$$

SAT Model: Clauses

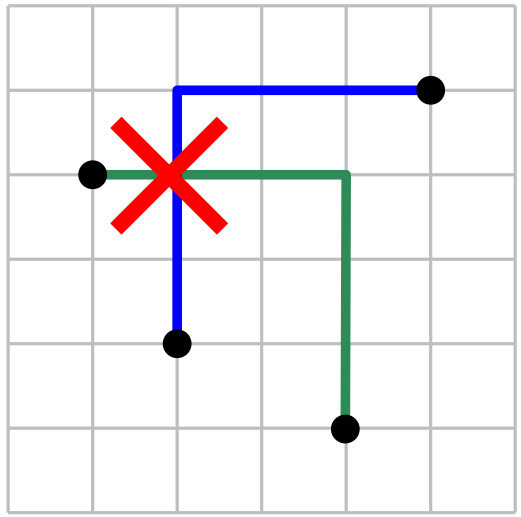
- Injective mapping V to P
- L-shaped edges:
  ab connects either vertically or horizontally to a (and b)
- No overlapping edges

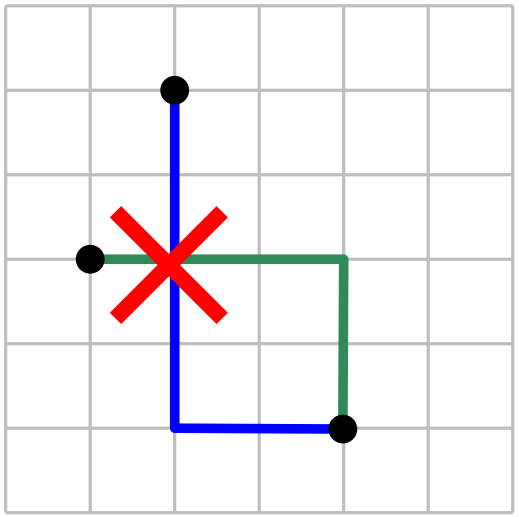
  If p left of q and r, ← only depends on point set
  and $v_a \to p$, $v_b \to q$, $v_c \to r$, ← only depends on variables
  then not both edges ab and ac horizonally connected

```
for p,q,r in permutations(range(n),3):
    for a,b,c in permutations(range(n),3):
        if p<q and p<r and (a,b) in E and (a,c) in E:
            CNF.append([-M(a,p),-M(b,q),-M(c,r),-H(a,b),-H(a,c)]
```

SAT Model: Clauses

- Injective mapping V to P
- L-shaped edges:
  ab connects either vertically or horizontally to a (and b)
- No overlapping edges
- No crossing edges

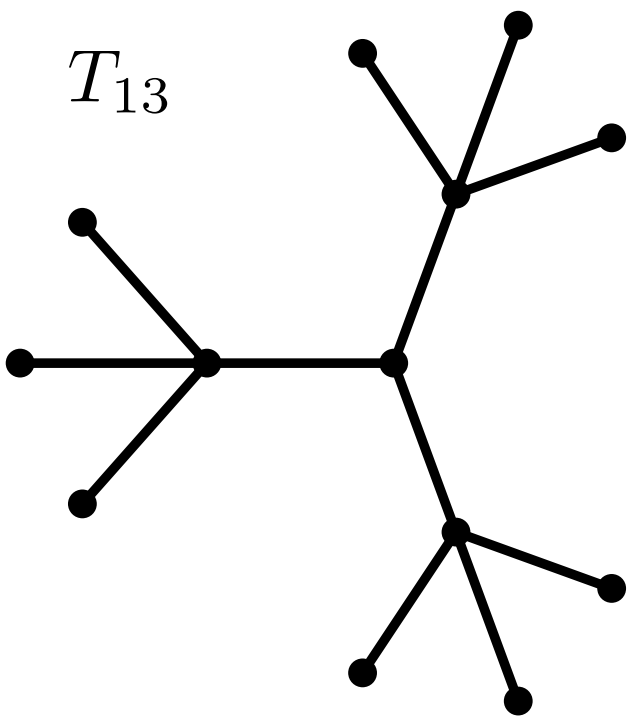




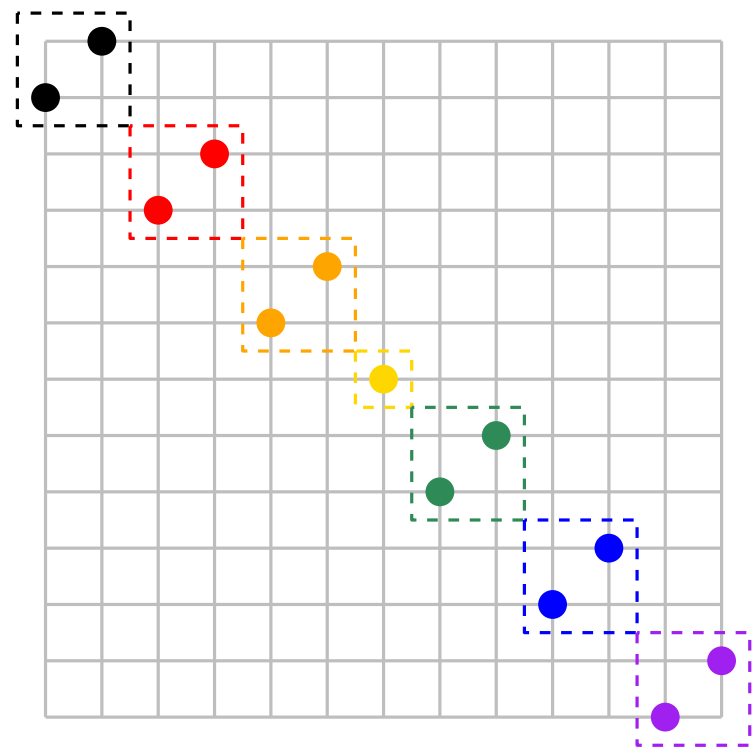
Results

- Every tree on $n \leq 12$ vertices admits an L-shaped embedding in every set of n points.
- T_{13} has no L-shaped embedding in P_{13}.

T_{13}



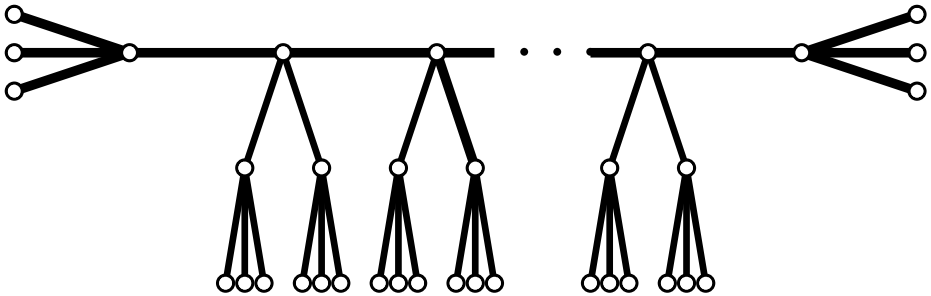
P_{13}

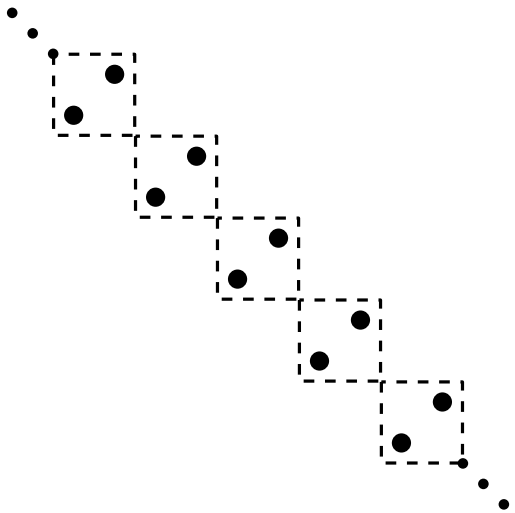


100+ cpu days

Results

- Every tree on $n \leq 12$ vertices admits an L-shaped embedding in every set of n points.
- T_{13} has no L-shaped embedding in P_{13}.
- further examples for $n \in \{13, 14, 16, 17, 18, 19, 20\}$
- <u>infinite</u> family if cyclic order fixed





And now for something completely different

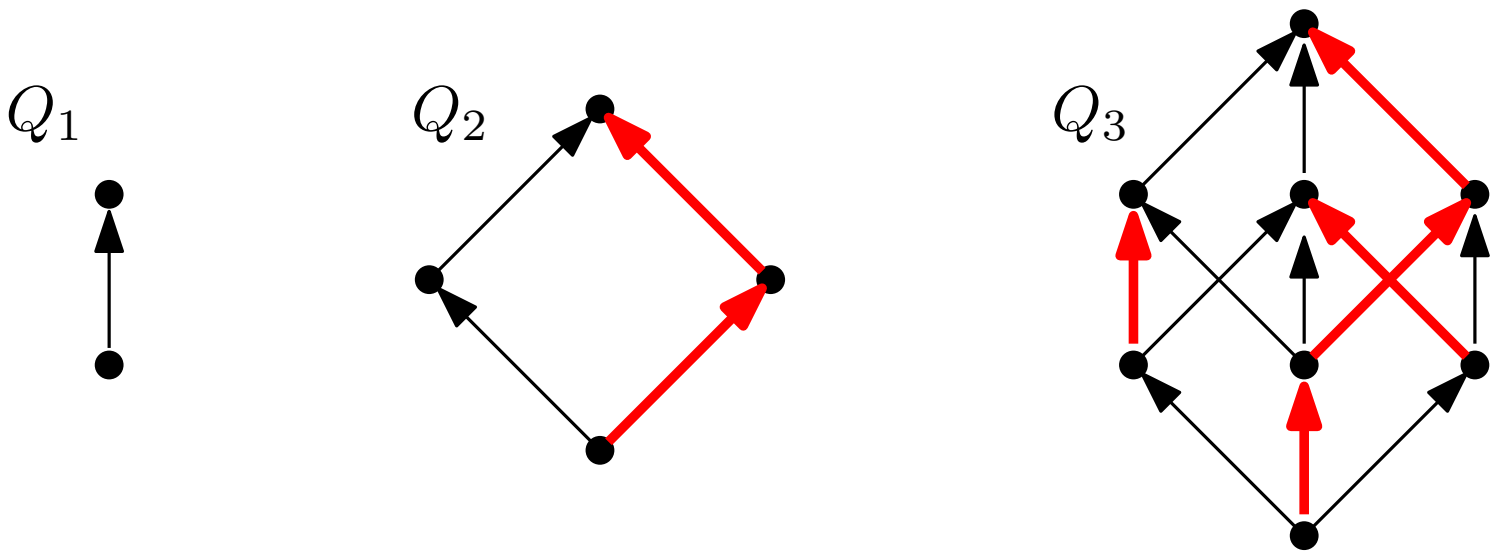
Orthogonal Chain Decompositions

joint work with
Karl Däubel, Sven Jäger, and Torsten Mütze

Orthogonal Chain Decompositions

- Boolean lattice Q_n

can be partitioned into $\binom{n}{\lfloor n/2 \rfloor}$ chains [Dilworth '50]



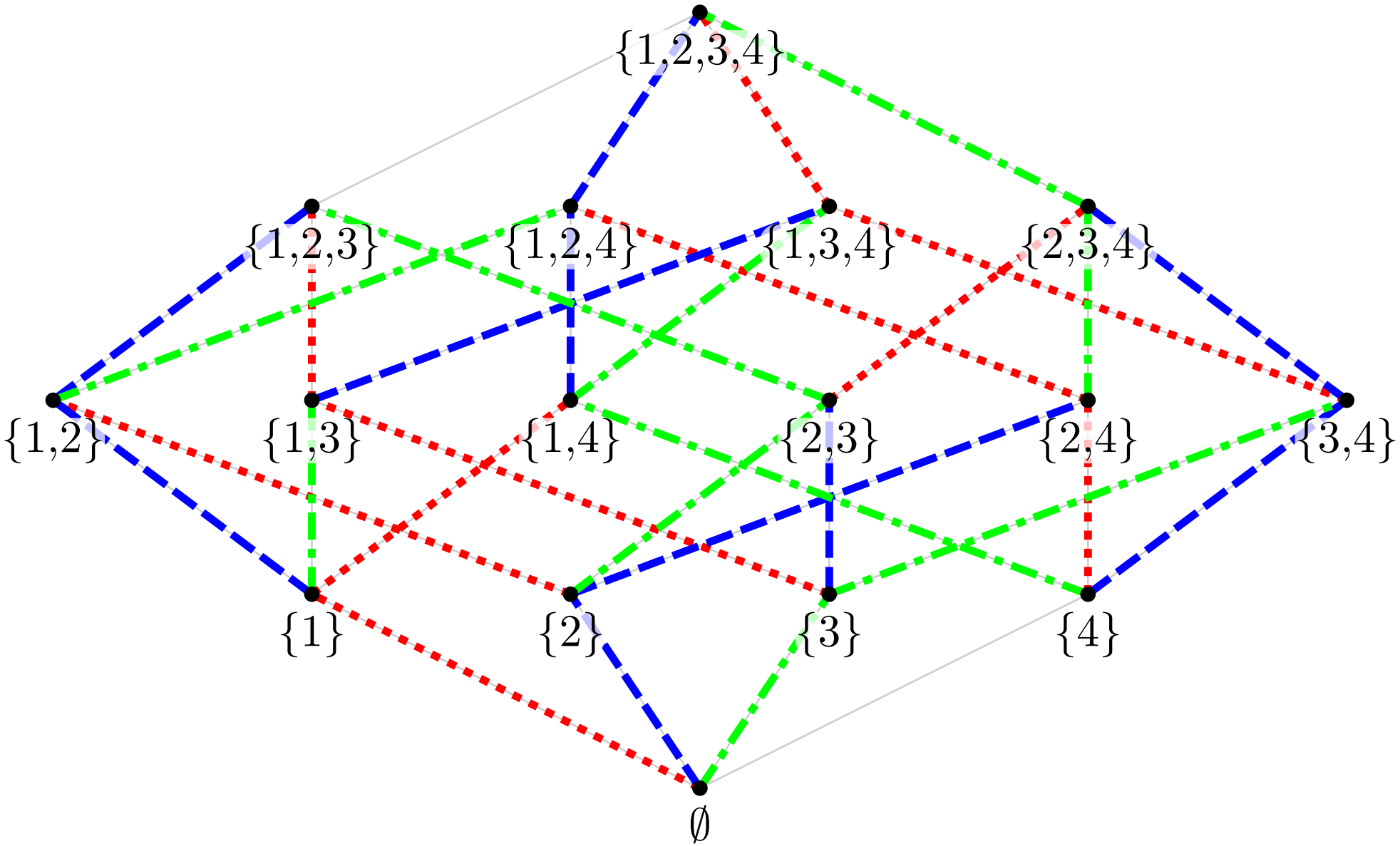
Orthogonal Chain Decompositions

- Boolean lattice Q_n

  can be partitioned into $\binom{n}{\lfloor n/2 \rfloor}$ chains [Dilworth '50]

- *orthogonal* chain-decompositions:
  any two chains from two decompositions
  have at most one element in common

Orthogonal Chain Decompositions



Orthogonal Chain Decompositions

- $\exists$ two orthogonal CDs in $Q_{n\geq 2}$ [Shearer-Kleitman '79]
- moreover, they conjectured $\lfloor n/2 \rfloor + 1$

Orthogonal Chain Decompositions

- $\exists$ two orthogonal CDs in $Q_{n\geq 2}$ [Shearer-Kleitman '79]
- moreover, they conjectured $\lfloor n/2 \rfloor + 1$
- $\exists$ three orthogonal CDs in $Q_{n\geq 24}$ [Spink '17]

Proof Idea:

1. (specific) orthogonal CDs for Q_5 and Q_7
2. Lemma "Q_a, $Q_b \Rightarrow Q_{a+b}$"

Orthogonal Chain Decompositions

- $\exists$ two orthogonal CDs in $Q_{n\geq 2}$ [Shearer-Kleitman '79]
- moreover, they conjectured $\lfloor n/2 \rfloor + 1$

- $\exists$ three orthogonal CDs in $Q_{n\geq 24}$ [Spink '17]

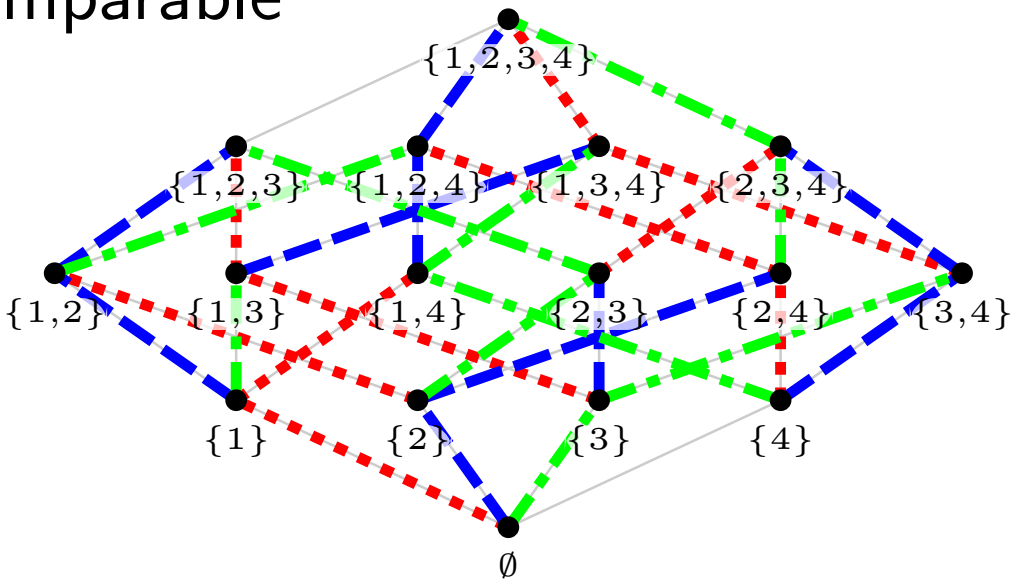
n	1	2	3	4	5	6	7	8	9	10	11
almost-orth.	1	2	2	2	3	3*	4*	3*	3*	3	4*
$\lfloor n/2 \rfloor + 1$	1	2	2	3	3	4	4	5	5	6	6

found via SAT solver

Theorem: $\exists$ four orthogonal CDs in $Q_{n\geq 60}$.
[Däubel-Jäger-Mütze-S. '18]

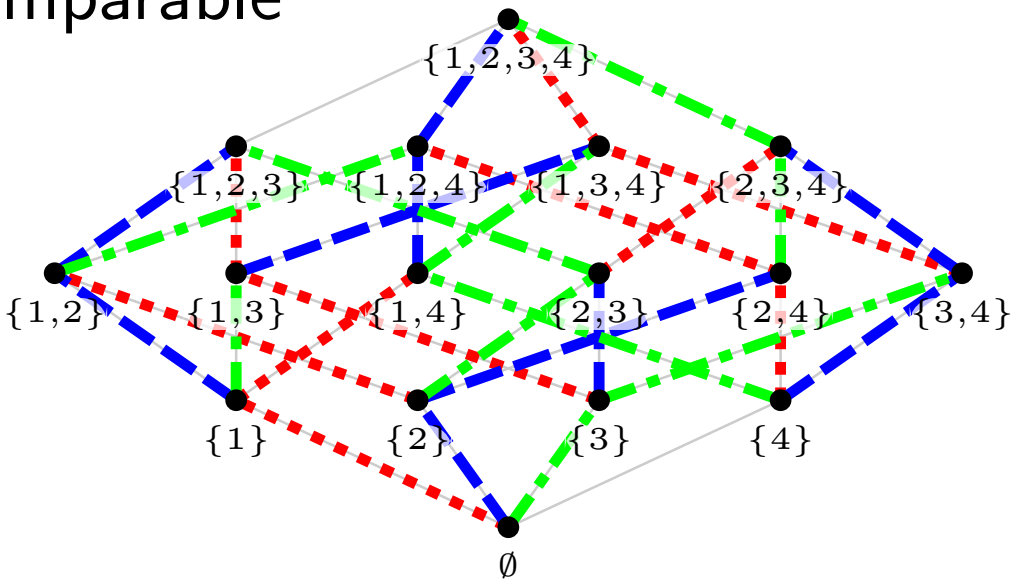
SAT Instance

- variables for element-chain assignment
- constraints enure orthogonal chain decomposition:
  - all elements in chain comparable
  - each $\binom{n}{n/2}$ chains form a decomposition
  - two chains overlap in at most 1 element



SAT Instance

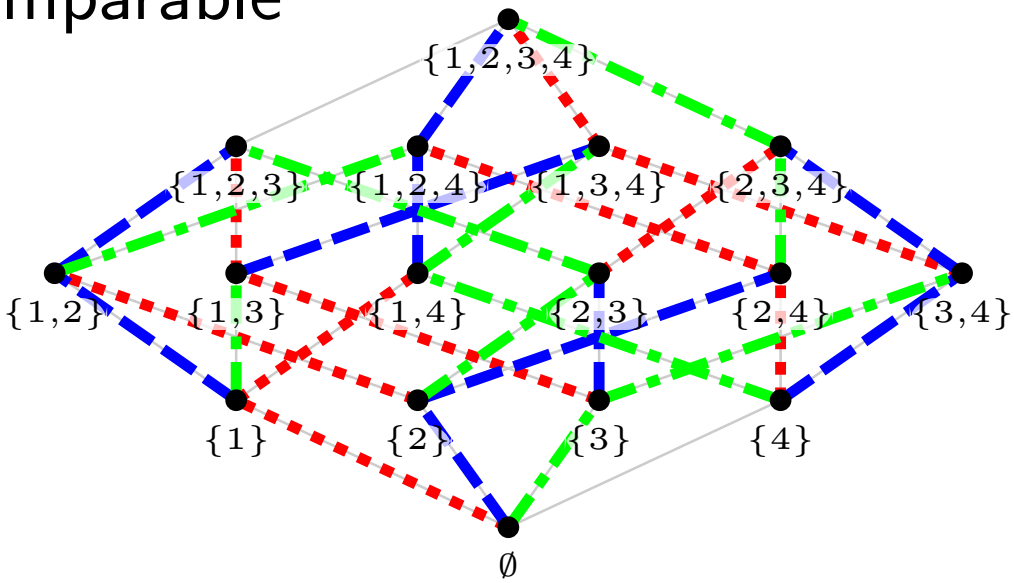
- variables for element-chain assignment
- constraints enure orthogonal chain decomposition:
  - all elements in chain comparable
  - each $\binom{n}{n/2}$ chains form a decomposition
  - two chains overlap in at most 1 element



- Problem: naive CNF formulation much too big

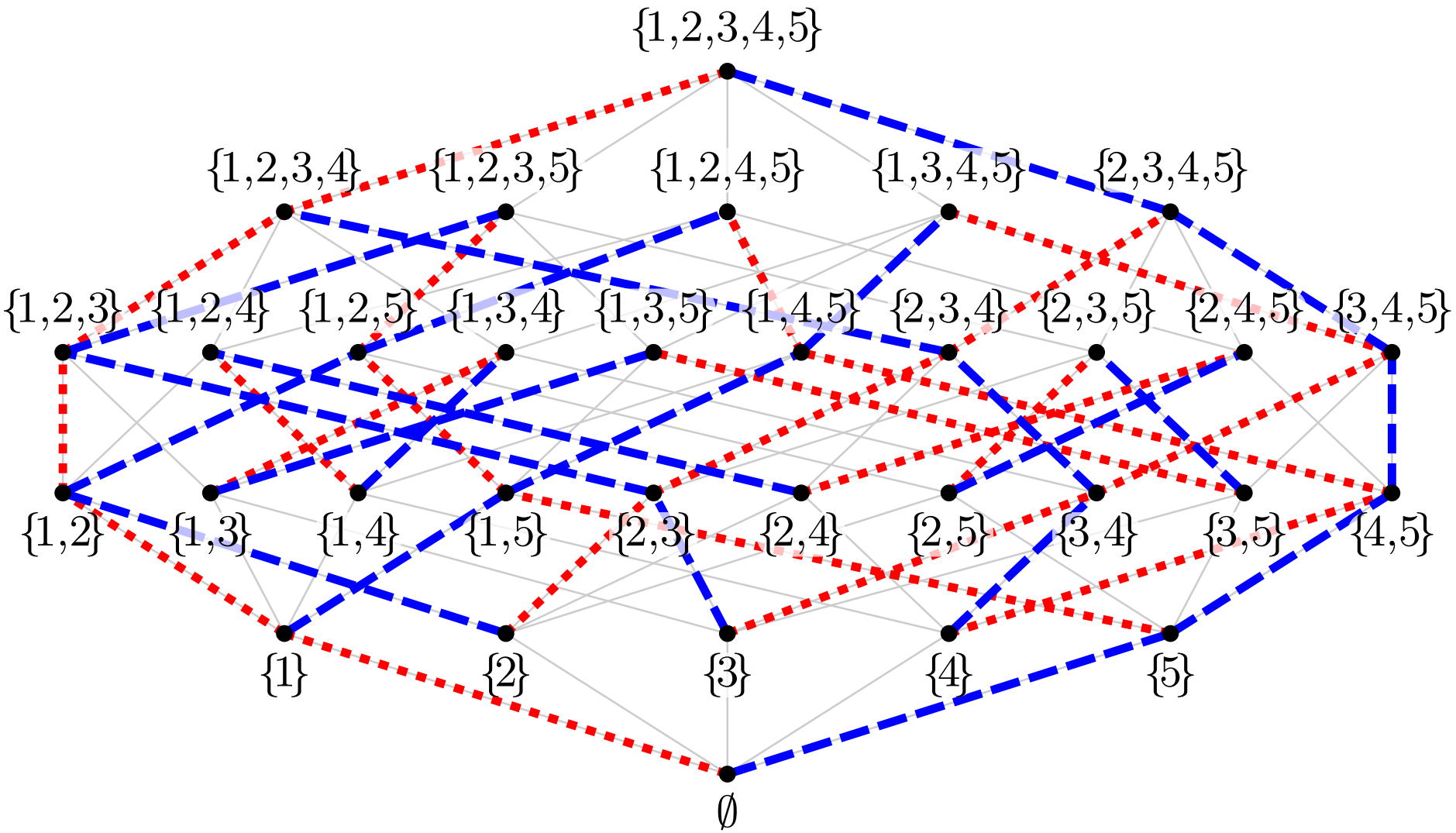
SAT Instance

- variables for element-chain assignment
- constraints enure orthogonal chain decomposition:
  - all elements in chain comparable
  - each $\binom{n}{n/2}$ chains form a decomposition
  - two chains overlap in at most 1 element

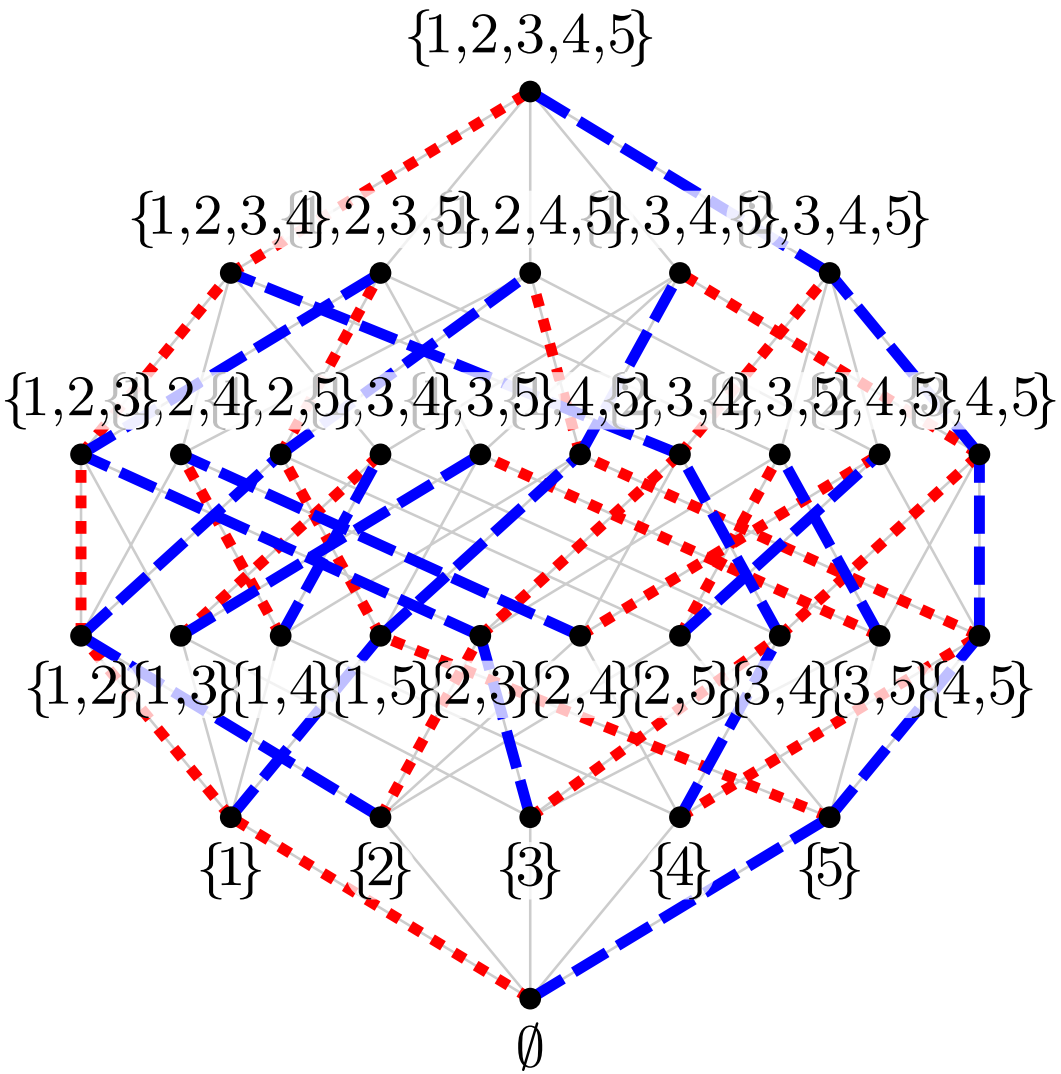


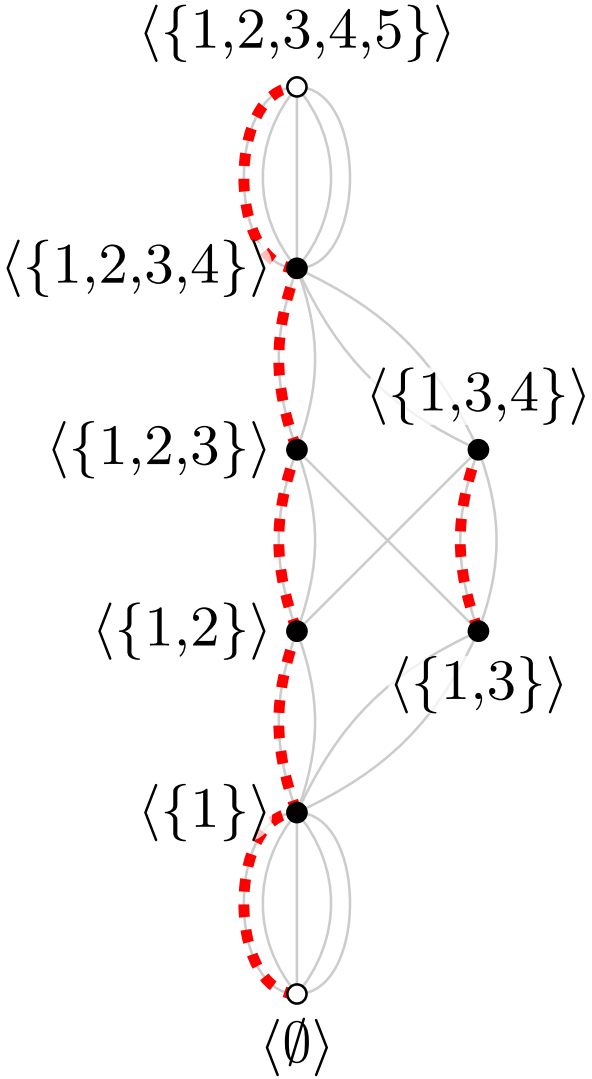
- Problem: naive CNF formulation much too big
- use symmetries

SAT Instance



SAT Instance





SAT Instance

- iteratively add clauses to “correct errors”

SAT Instance

- iteratively add clauses to “correct errors”
- 2023: cut branching

And now for something more completely different

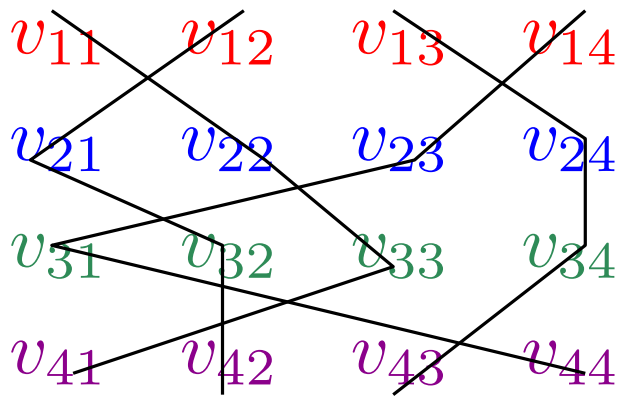
Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

$$
\begin{matrix}
v_{11} & v_{12} & v_{13} & v_{14} \\
v_{21} & v_{22} & v_{23} & v_{24} \\
v_{31} & v_{32} & v_{33} & v_{34} \\
v_{41} & v_{42} & v_{43} & v_{44}
\end{matrix}
$$

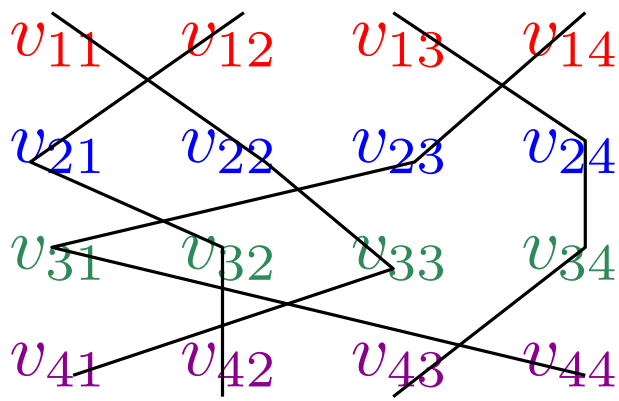
Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

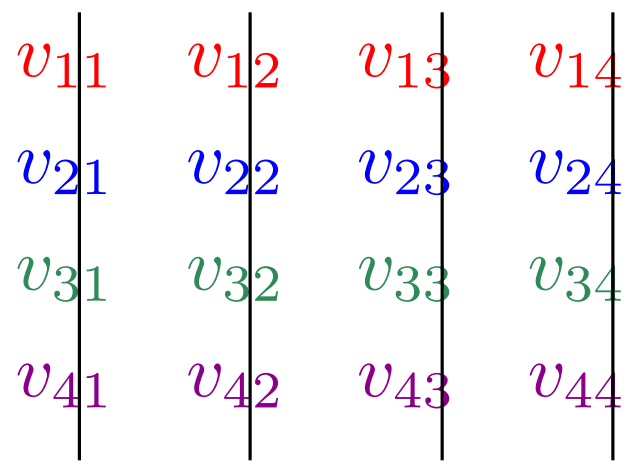


Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

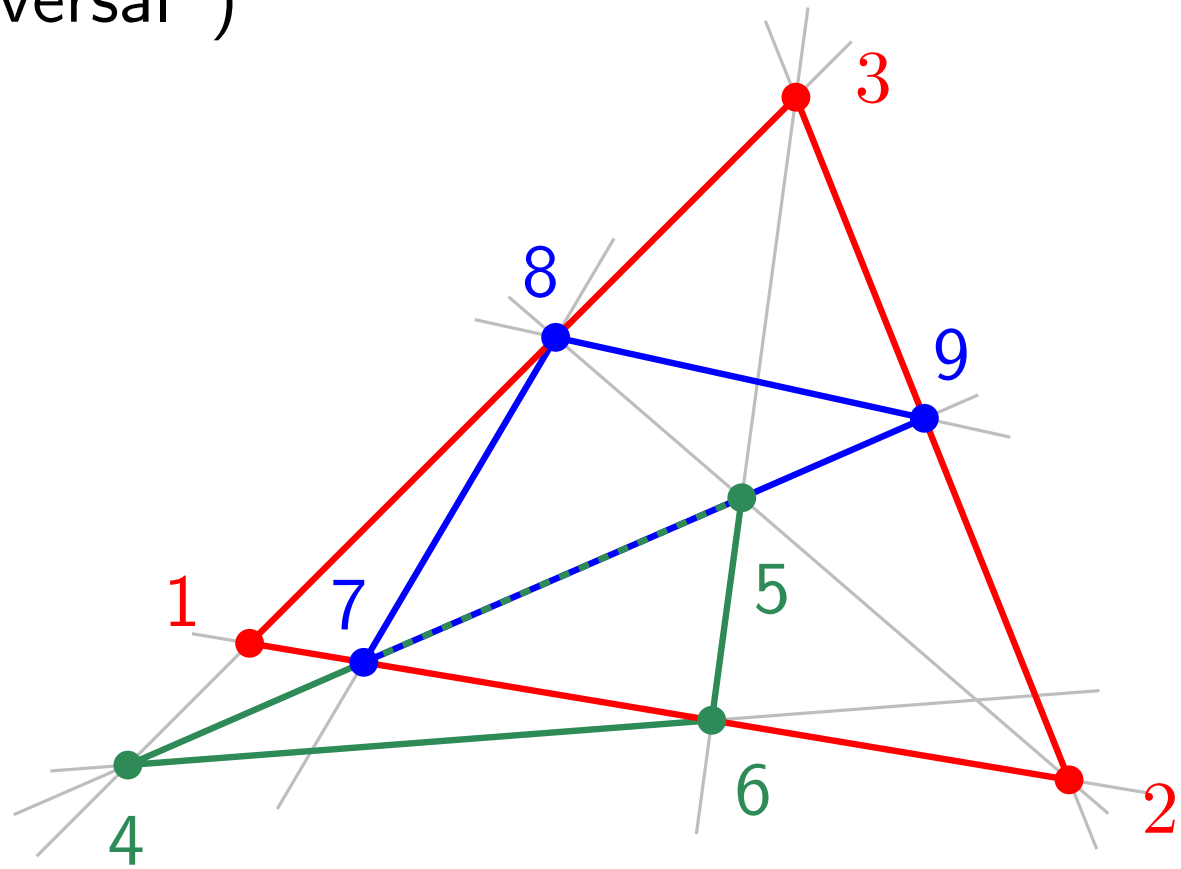


w.l.o.g. (relabeling)



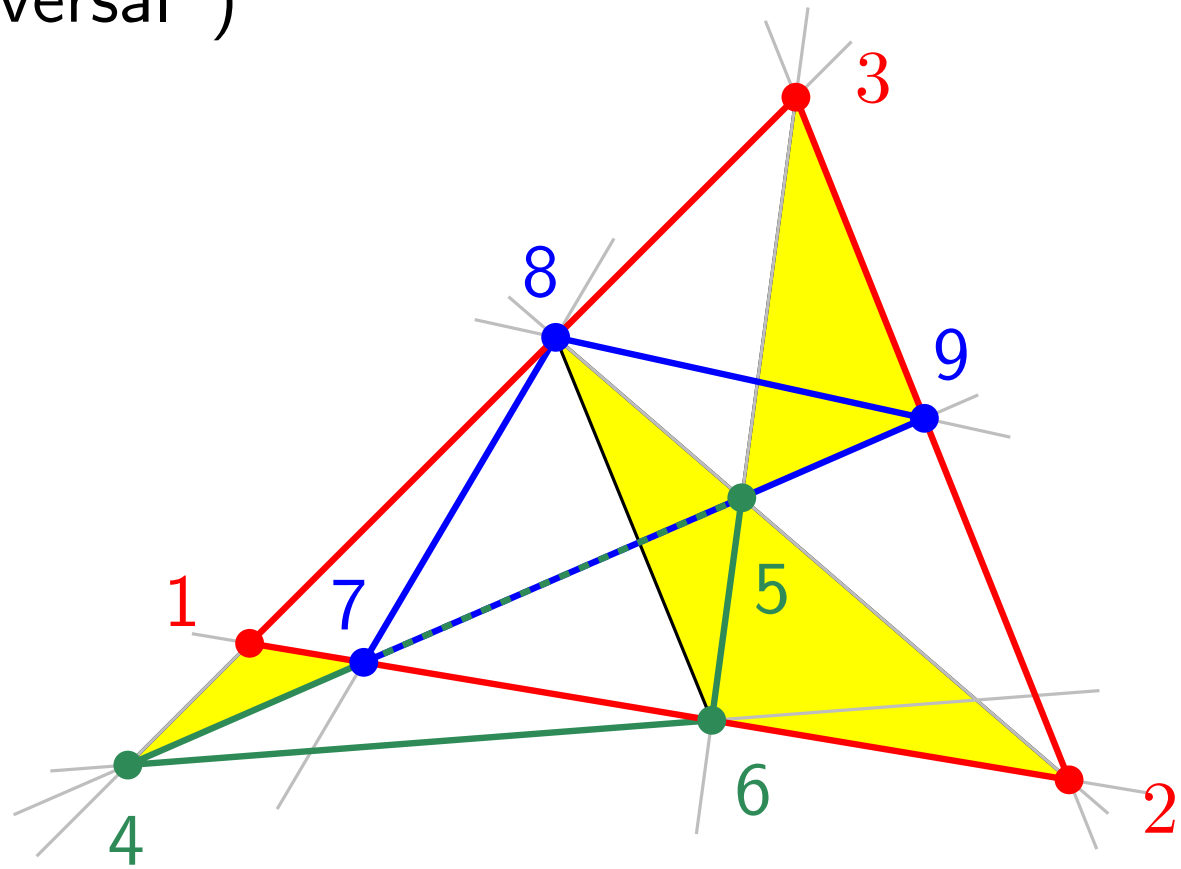
Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")



Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")



Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

Geelen and Humphries '06: paving matroids (girth r)

Friedman and McGuinness '20: girth $> r - o(\sqrt{r})$

Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

Geelen and Humphries '06: paving matroids (girth r)

Friedman and McGuinness '20: girth $> r - o(\sqrt{r})$

Kirchweger S. Szeider '22: $r \leq 4$; $r = 5$ with girth ≥ 4

Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

Geelen and Humphries '06: paving matroids (girth r)

Friedman and McGuinness '20: girth $> r - o(\sqrt{r})$

Kirchweger S. Szeider '22: $r \leq 4$; $r = 5$ with girth ≥ 4

Bucic '20: $\frac{r}{2}(1 - o(1))$ transversals

Pokrovskiy '20: $r(1 - o(1))$ rainbow indep. sets of card. $r(1 - o(1))$

Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

$$\begin{array}{cccc} v_{11} & v_{12} & v_{13} & v_{14} \\ v_{21} & v_{22} & v_{23} & v_{24} \\ v_{31} & v_{32} & v_{33} & v_{34} \\ v_{41} & v_{42} & v_{43} & v_{44} \end{array}$$

w.l.o.g. (relabeling)

$$\begin{array}{cccc} v_{11} & v_{12} & v_{13} & v_{14} \\ v_{21} & v_{22} & v_{23} & v_{24} \\ v_{31} & v_{32} & v_{33} & v_{34} \\ v_{41} & v_{42} & v_{43} & v_{44} \end{array}$$

Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

$$\begin{array}{cccc} v_{11} & v_{12} & v_{13} & v_{14} \\ v_{21} & v_{22} & v_{23} & v_{24} \\ v_{31} & v_{32} & v_{33} & v_{34} \\ v_{41} & v_{42} & v_{43} & v_{44} \end{array}$$

w.l.o.g. (relabeling)

$$\begin{array}{cccc} v_{11} & v_{12} & v_{13} & v_{14} \\ v_{21} & v_{22} & v_{23} & v_{24} \\ v_{31} & v_{32} & v_{33} & v_{34} \\ v_{41} & v_{42} & v_{43} & v_{44} \end{array}$$

Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

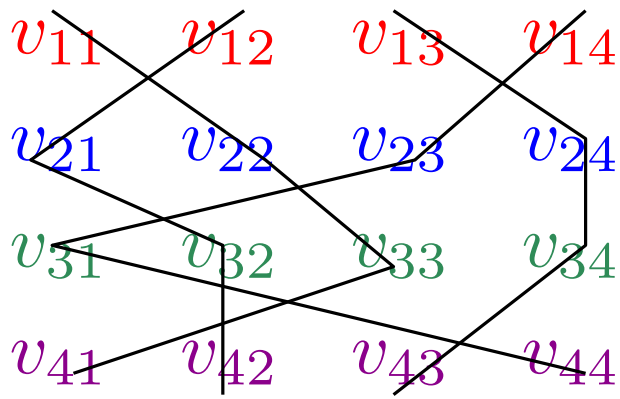
$$\begin{array}{cccc} v_{11} & v_{12} & v_{13} & v_{14} \\ v_{21} & v_{22} & v_{23} & v_{24} \\ v_{31} & v_{32} & v_{33} & v_{34} \\ v_{41} & v_{42} & v_{43} & v_{44} \end{array}$$

w.l.o.g. (relabeling)

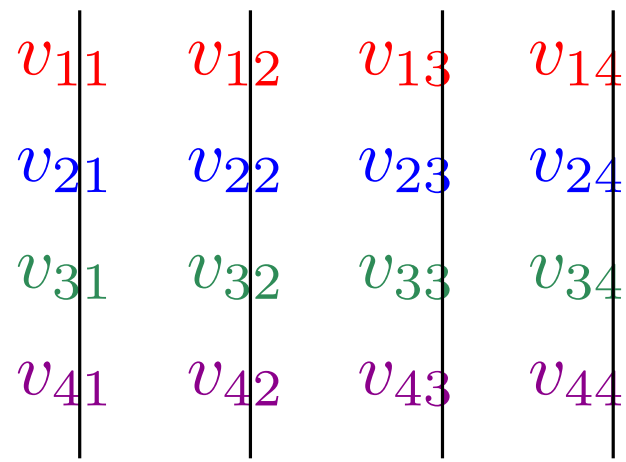
$$\begin{array}{cccc} v_{11} & v_{12} & v_{13} & v_{14} \\ v_{21} & v_{22} & v_{23} & v_{24} \\ v_{31} & v_{32} & v_{33} & v_{34} \\ v_{41} & v_{42} & v_{43} & v_{44} \end{array}$$

Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")

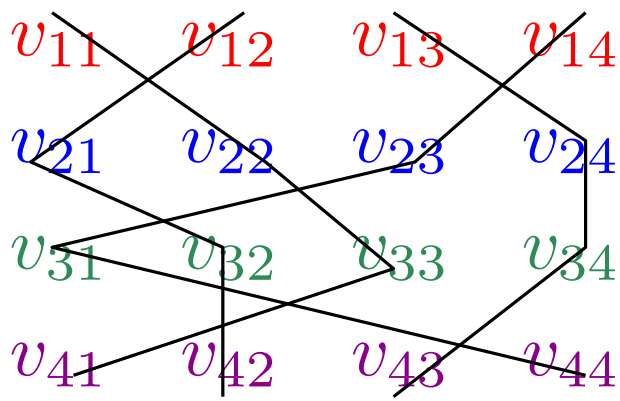


w.l.o.g. (relabeling)

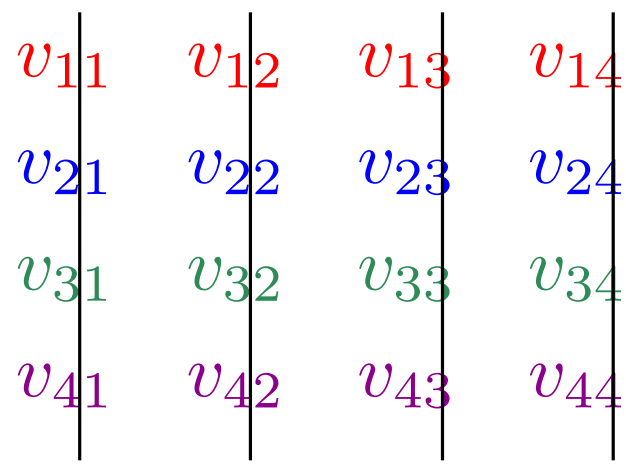


Rota's Basis Conjecture

Conjecture: $r \times r$ elements such each row is basis
$\Rightarrow \exists$ permution of each row s.t. each column is basis
("transversal")



w.l.o.g. (relabeling)

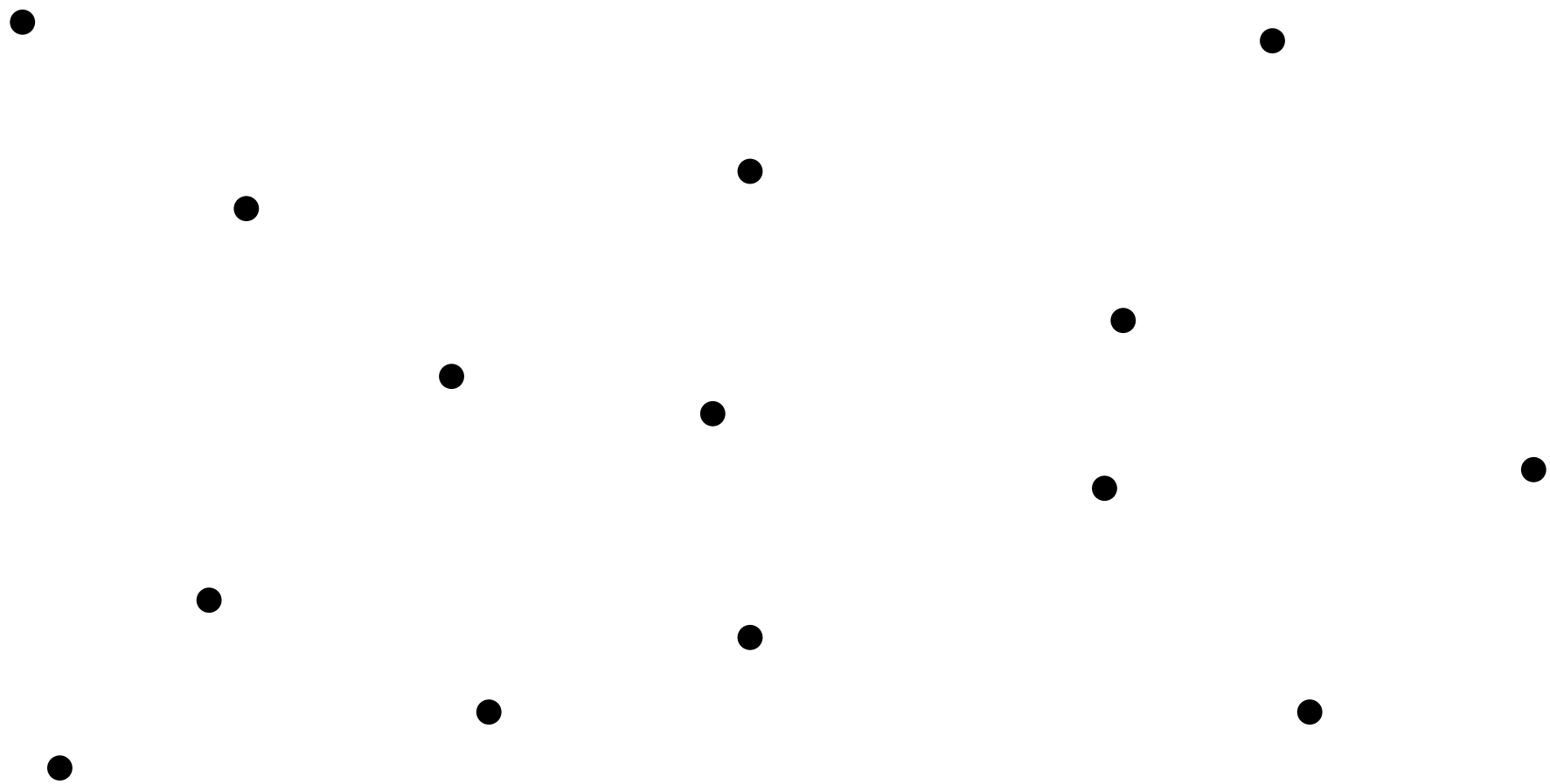


use SAT to check that $k < r$ transversals cannot occur

And now for something more completely different

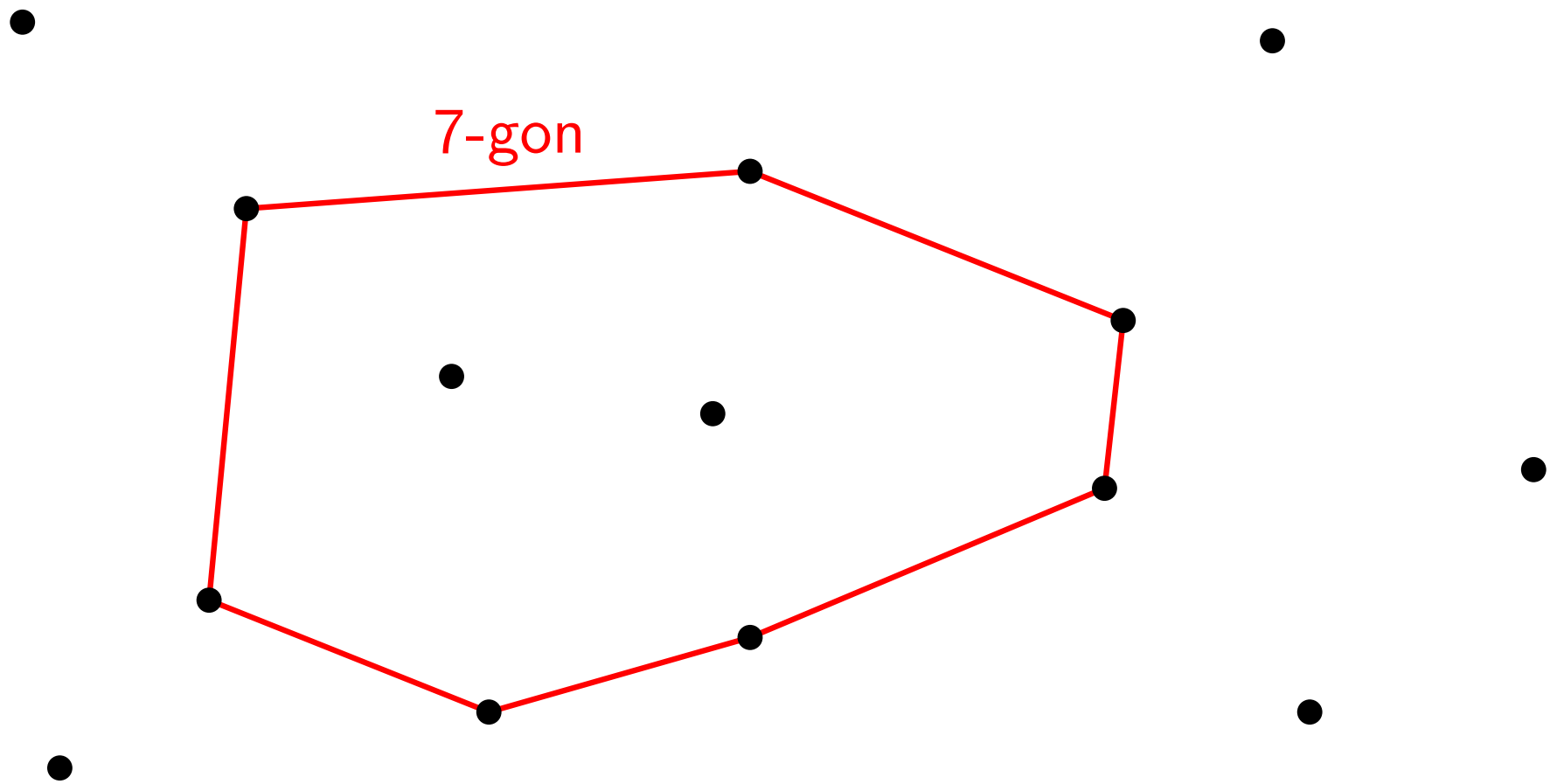
Classical Erdős–Szekeres

- Given n points in the plane in general position, is there subset size k in convex position ("k-gon")?



Classical Erdős–Szekeres

- Given n points in the plane in general position, is there subset size k in convex position ("k-gon")?



Classical Erdős–Szekeres

- Given n points in the plane in general position, is there subset size k in convex position ("k-gon")?

Theorem. $2^{k-2} + 1 \leq g(k) \leq \binom{2k-4}{k-2}$. [Erdős–Szekeres '35]

equality conjectured by Szekeres, Erdős offered 500$ for a proof

Classical Erdős–Szekeres

- Given n points in the plane in general position, is there subset size k in convex position ("k-gon")?

Theorem. $2^{k-2} + 1 \leq g(k) \leq \binom{2k-4}{k-2}$. [Erdős–Szekeres '35]

$\vdots$

Theorem. $g(k) \leq 2^{k+o(k)}$. [Suk '17]

Classical Erdős–Szekeres

- Given n points in the plane in general position, is there subset size k in convex position ("k-gon")?

Theorem. $2^{k-2} + 1 \leq g(k) \leq \binom{2k-4}{k-2}$. [Erdős–Szekeres '35]

$\vdots$

Theorem. $g(k) \leq 2^{k+o(k)}$. [Suk '17]

Known: $g(4) = 5$, $g(5) = 9$, $g(6) = 17$

computer assisted proof, 1500 CPU hours [Szekeres–Peters '06]

Classical Erdős–Szekeres

- Given n points in the plane in general position, is there subset size k in convex position ("k-gon")?

Theorem. $2^{k-2} + 1 \leq g(k) \leq \binom{2k-4}{k-2}$. [Erdős–Szekeres '35]

$\vdots$

Theorem. $g(k) \leq 2^{k+o(k)}$. [Suk '17]

10 CPU minutes using SAT!

Known: $g(4) = 5$, $g(5) = 9$, $g(6) = 17$

computer assisted proof, 1500 CPU hours [Szekeres–Peters '06]

Point Configurations

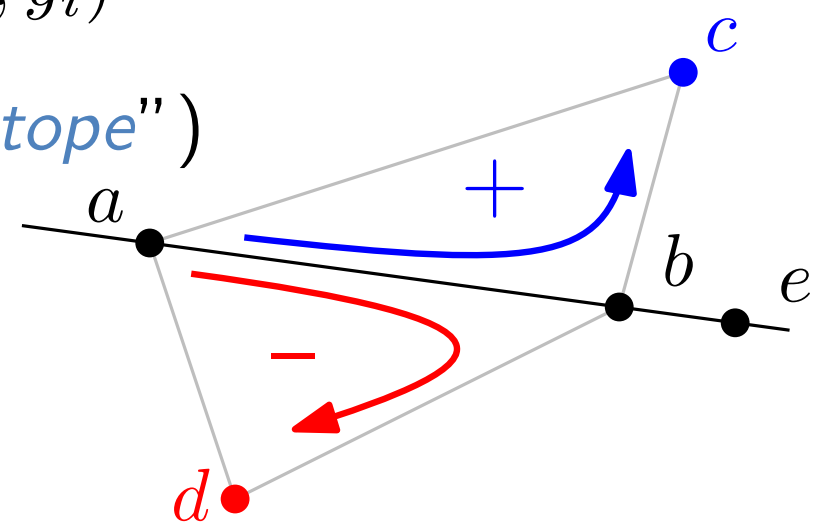
Point Configurations

$P = \{p_1, \ldots, p_n\} \subset \mathbb{R}^2$, $p_i = (x_i, y_i)$

induces triple-orientations ("*chirotope*")

$\chi : [n]^3 \to \{+, 0, -\}$

"positive, collinear, negative"



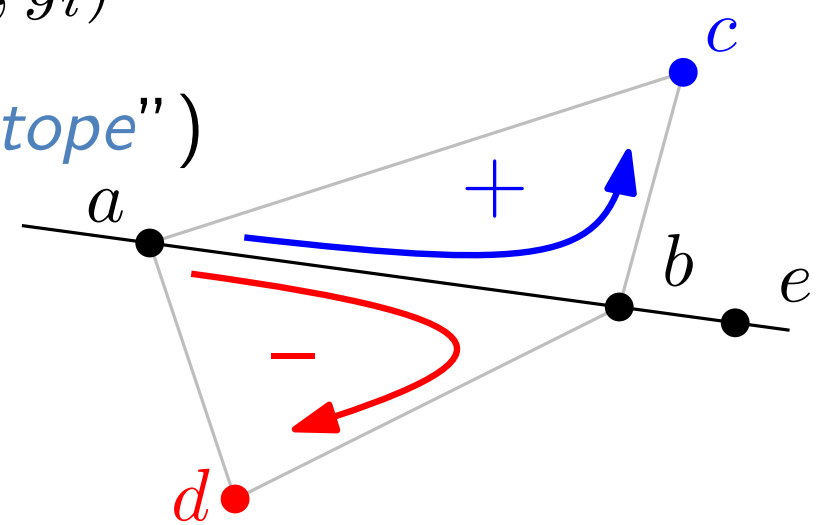
Point Configurations

$P = \{p_1, \ldots, p_n\} \subset \mathbb{R}^2$, $p_i = (x_i, y_i)$

induces triple-orientations ("*chirotope*")

$\chi : [n]^3 \to \{+, 0, -\}$

"positive, collinear, negative"



formally:

$$\chi(a, b, c) := \operatorname{sgn} \det \begin{pmatrix} 1 & 1 & 1 \\ x_a & x_b & x_c \\ y_a & y_b & y_c \end{pmatrix}$$

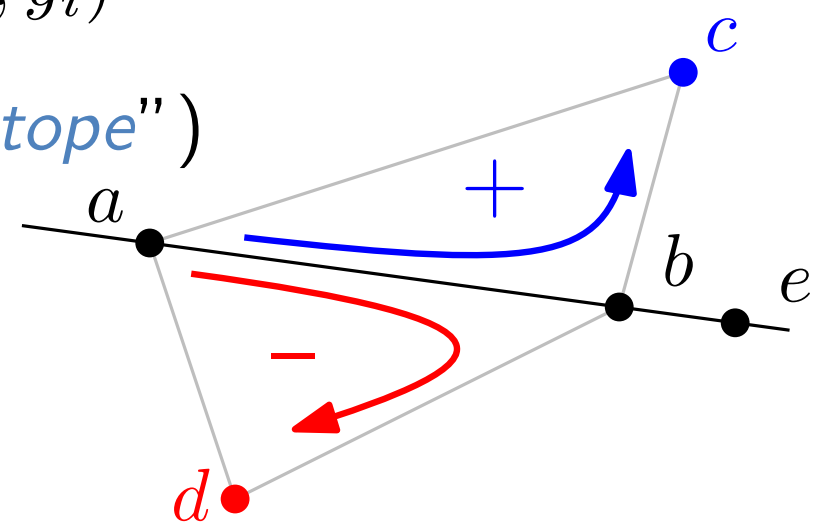
Point Configurations

$P = \{p_1, \ldots, p_n\} \subset \mathbb{R}^2$, $p_i = (x_i, y_i)$

induces triple-orientations ("*chirotope*")

$\chi : [n]^3 \to \{+, 0, -\}$

"positive, collinear, negative"



1.) *alternating axioms*:

$\chi(a, b, c) = \chi(b, c, a), \quad \chi(a, b, c) = -\chi(c, b, a)$

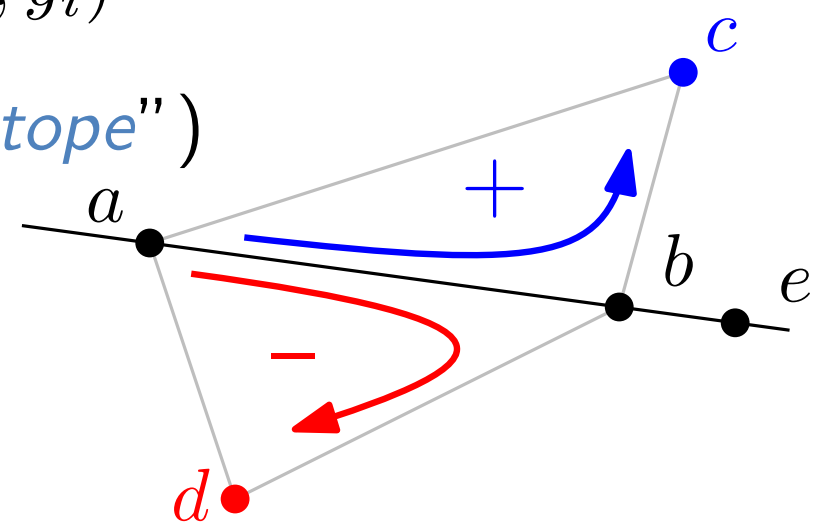
Point Configurations

$P = \{p_1, \ldots, p_n\} \subset \mathbb{R}^2$, $p_i = (x_i, y_i)$

induces triple-orientations ("*chirotope*")

$\chi : [n]^3 \to \{+, 0, -\}$

"positive, collinear, negative"



1.) *alternating axioms*:

$\chi(a,b,c) = \chi(b,c,a), \quad \chi(a,b,c) = -\chi(c,b,a)$

2.) *exchange axioms*:

$$\left.\begin{array}{l} \chi(dbc)\cdot\chi(aef) > 0 \\ \chi(ebc)\cdot\chi(daf) > 0 \\ \chi(fbc)\cdot\chi(dea) > 0 \end{array}\right\} \Rightarrow \chi(abc)\cdot\chi(def) > 0$$

Point Configurations

$$\det(x_1, \ldots, x_r) \cdot \det(y_1, \ldots, y_r) =$$

$$\sum_{i=1}^{r} \det(y_i, x_2, \ldots, x_r) \cdot \det(y_1, \ldots, y_{i-1}, x_1, y_{i+1}, \ldots, y_r)$$

2.) *exchange axioms*: (via Grassman-Plücker relations)

$$\left.\begin{array}{l} \chi(dbc) \cdot \chi(aef) > 0 \\ \chi(ebc) \cdot \chi(daf) > 0 \\ \chi(fbc) \cdot \chi(dea) > 0 \end{array}\right\} \Rightarrow \chi(abc) \cdot \chi(def) > 0$$

Point Configurations

$$\det(\overset{a}{x_1}, \overset{b}{\ldots}, \overset{c}{x_r}) \cdot \det(\overset{d}{y_1}, \overset{e}{\ldots}, \overset{f}{y_r}) =$$

$$\sum_{i=1}^{r} \det(y_i, x_2, \ldots, x_r) \cdot \det(y_1, \ldots, y_{i-1}, x_1, y_{i+1}, \ldots, y_r)$$

2.) *exchange axioms*: (via Grassman-Plücker relations)

$$\left.\begin{array}{l} \chi(dbc) \cdot \chi(aef) > 0 \\ \chi(ebc) \cdot \chi(daf) > 0 \\ \chi(fbc) \cdot \chi(dea) > 0 \end{array}\right\} \Rightarrow \chi(abc) \cdot \chi(def) > 0$$

Point Configurations

(via Laplace expansion)

$$\det(x_1, \ldots, x_r) \cdot \det(y_1, \ldots, y_r) =$$

$$\sum_{i=1}^{r} \det(y_i, x_2, \ldots, x_r) \cdot \det(y_1, \ldots, y_{i-1}, x_1, y_{i+1}, \ldots, y_r)$$

2.) *exchange axioms*:

(via Grassman-Plücker relations)

$$\left.\begin{array}{l} \chi(dbc) \cdot \chi(aef) > 0 \\ \chi(ebc) \cdot \chi(daf) > 0 \\ \chi(fbc) \cdot \chi(dea) > 0 \end{array}\right\} \Rightarrow \chi(abc) \cdot \chi(def) > 0$$

Point Configurations

alternating and exchange axioms are necessary conditions
(problem: realizability of chirotopes ETR-hard)

Point Configurations

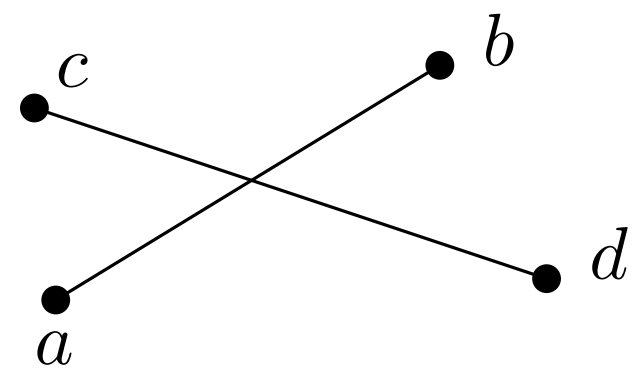
alternating and exchange axioms are necessary conditions
(problem: realizability of chirotopes ETR-hard)

exchange axioms give $\Theta(n^6)$ constraints,
$\Theta(n^5)$ of them sufficient

SAT Formulation

- Variables for triple-orientations: $\chi_{abc} \in \{+, -\}$
- Axiomatize "point set": chirotope/signotope axioms
- Crossings via triple-orientations

$\chi_{abc} \neq \chi_{abd}$ and $\chi_{cda} \neq \chi_{cdb}$



SAT Formulation

- Variables for triple-orientations: $\chi_{abc} \in \{+, -\}$
- Axiomatize "point set": chirotope/signotope axioms
- Crossings via triple-orientations
- k-gons via crossings

  (every 4-tuple in k-gon is in convex positon)

And now for something more completely different

Universal Point Sets for Planar Graphs

joint work with
Hendrik Schrezenmaier and Raphael Steiner

Universal Point Sets for Planar Graphs

Definition: *n-universal* point set S:
$\forall$ planar n-vertex graph G can be drawn straight-line on S.

Universal Point Sets for Planar Graphs

Definition: *n-universal* point set S:
$\forall$ planar n-vertex graph G can be drawn straight-line on S.

- $n \times n$ grid is n-universal [De Fraysseix–Pach–Pollack '90]
- ...
- $|S| \leq \frac{n^2}{4} - O(n)$ [Bannister–Cheng–Devanny–Eppstein '14]

Universal Point Sets for Planar Graphs

Definition: *n-universal* point set S:
$\forall$ planar n-vertex graph G can be drawn straight-line on S.

- $n \times n$ grid is n-universal [De Fraysseix–Pach–Pollack '90]
- ...
- $|S| \leq \frac{n^2}{4} - O(n)$ [Bannister–Cheng–Devanny–Eppstein '14]
- $|S| \geq n + \Omega(\sqrt{n})$ [De Fraysseix–Pach–Pollack '90]
- ...
- $|S| \geq 1.235n(1 + o(1))$ [Kurowski '04]

Universal Point Sets for Planar Graphs

Definition: *n-universal* point set S:
$\forall$ planar n-vertex graph G can be drawn straight-line on S.

- $n \times n$ grid is n-universal [De Fraysseix–Pach–Pollack '90]
- ...
- $|S| \leq \frac{n^2}{4} - O(n)$ [Bannister–Cheng–Devanny–Eppstein '14]
- $|S| \geq n + \Omega(\sqrt{n})$ [De Fraysseix–Pach–Pollack '90]
- ...
- $|S| \geq 1.235n(1 + o(1))$ [Kurowski '04]

- $\exists$ n-universal sets for $n \leq 10$ [Cardinal–Hoffmann–Kusters '15]
- $\nexists$ n-universal sets for $n \geq 15$ [Cardinal–Hoffmann–Kusters '15]

Universal Point Sets for Planar Graphs

Definition: *n-universal* point set S:
$\forall$ planar n-vertex graph G can be drawn straight-line on S.

- $n \times n$ grid is n-universal [De Fraysseix–Pach–Pollack '90]
- ...
- $|S| \leq \frac{n^2}{4} - O(n)$ [Bannister–Cheng–Devanny–Eppstein '14]
- $|S| \geq n + \Omega(\sqrt{n})$ [De Fraysseix–Pach–Pollack '90]
- ...
- $|S| \geq 1.235n(1 + o(1))$ [Kurowski '04]

$|S| \geq 1.293n(1 + o(1))$

$\nexists$ 11-universal set on 11 points

- $\exists$ n-universal sets for $n \leq 10$ [Cardinal–Hoffmann–Kusters '15]
- $\nexists$ n-universal sets for $n \geq 15$ [Cardinal–Hoffmann–Kusters '15]

Universal Point Sets for Planar Graphs

SAT model for a fixed set S and fixed graph $G \in \mathcal{G}$:

- Injective mapping $V(G) \to S$
- No two edges cross (planarity)

(similar idea as in L-shaped model)

Universal Point Sets for Planar Graphs

SAT model for a fixed set S and fixed graph $G \in \mathcal{G}$:

- Injective mapping $V(G) \to S$
- No two edges cross (planarity)

(similar idea as in L-shaped model)

All in one SAT instance:

- all graphs can be handled simultaneously
- all point sets simultaneously (chirotope/signotope axioms)

. . . but solvers do not terminate . . .

Universal Point Sets for Planar Graphs

- Enumerate all triangulations on 11 vertices (plantri) (1,249)

Universal Point Sets for Planar Graphs

- Enumerate all triangulations on 11 vertices (plantri) (1,249)
- Enumerate all chirotopes on 11 points (2,343,203,071)

via chirotope axioms, 20 CPU hours, 100 GB storage
(or order type database by Aichholzer, Aurenhammer and Krasser)

Universal Point Sets for Planar Graphs

- Enumerate all triangulations on 11 vertices (plantri) (1,249)
- Enumerate all chirotopes on 11 points (2,343,203,071)

- Start with $\mathcal{G}$ as 11-vertex triangulations with maximum degree 10
- Test each pair S and G (SAT)

priority queue ("hard" triangulations first)

Universal Point Sets for Planar Graphs

- Enumerate all triangulations on 11 vertices (plantri) (1,249)
- Enumerate all chirotopes on 11 points (2,343,203,071)

- Start with $\mathcal{G}$ as 11-vertex triangulations with maximum degree 10
- Test each pair S and G (SAT)

- For remaining $\mathcal{G}$-universal sets, use IP to find minimal set of triangulations which need to be added

Universal Point Sets for Planar Graphs

- Enumerate all triangulations on 11 vertices (plantri) (1,249)
- Enumerate all chirotopes on 11 points (2,343,203,071)

- Start with $\mathcal{G}$ as 11-vertex triangulations with maximum degree 10
- Test each pair S and G (SAT)

- For remaining $\mathcal{G}$-universal sets, use IP to find minimal set of triangulations which need to be added

- 500 CPU days later:

conflict collection of 49 $n = 11$ stacked triangulations!

previously: 7393 on $n = 35$

Thank you for your attention!